# AN AUTOMATED RISK MITIGATION IN HEALTHCARE SOFTWARE DELIVERY USING CI&CD PIPELINES

**A PROJECT REPORT**

***Submitted by***

**SALAI SANMARGAM J (810022104072)**

**SATHISH S (810022104075)**

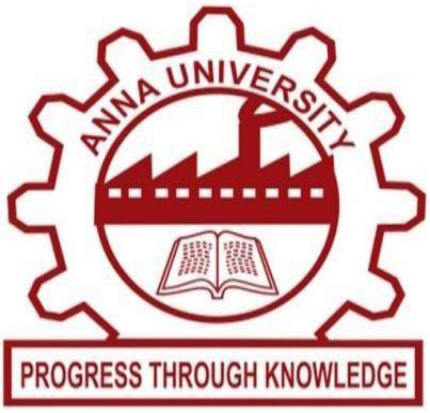
***in partial fulfillment for the award of the degree***

***of***

**BACHELOR OF ENGINEERING**

**IN**

**COMPUTER SCIENCE AND ENGINEERING**




**UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS,**
**TIRUCHIRAPPALLI- 620 024**

**ANNA UNIVERSITY:: CHENNAI 600 025**

**MAY 2026**

# ANNA UNIVERSITY : CHENNAI 600 025

## BONAFIDE CERTIFICATE

Certified that this project report " **AN AUTOMATED RISK MITIGATION IN HEALTHCARE SOFTWARE DELIVERY USING CI&CD PIPELINES** " is the bonafide work of " **SALAI SANMARGAM J (810022104072) AND SATHISH S (810022104075)** " who carried out the project work under my supervision.

**SIGNATURE**

**Dr. G. ANNAPOORANI**
**HEAD OF THE DEPARTMENT**
Assistant Professor (Sl. Gr)
Department of CSE/IT
University College of Engineering,
BIT Campus, Anna University
Tiruchirappalli – 620 024.

**SIGNATURE**

**Dr. P. KARTHIKEYAN**
**SUPERVISOR**
Assistant Professor
Department of CSE
University College of Engineering,
BIT Campus, Anna University
Tiruchirappalli – 620 024.

Submitted for the VIVA-VOCE to be held on _____

**INTERNAL EXAMINER** **EXTERNAL EXAMINER**

# DECLARATION

We hereby declare that the work entitled "**AN AUTOMATED RISK MITIGATION IN HEALTHCARE SOFTWARE DELIVERY USING CI&CD PIPELINES**" is submitted in partial fulfilment of the requirement for the award of the degree in B.E. – Computer Science and Engineering in University College of Engineering, BIT Campus, Anna University, Tiruchirappalli. It is the record of our own work carried out during the academic year 2025-2026 under the supervision and guidance of **Dr. P. KARTHIKEYAN**, Assistant Professor, Department of Computer Science and Engineering, University College of Engineering, BIT Campus, Anna University, Tiruchirappalli. The extent and source of information are derived from the existing literature and have been indicated through the dissertation at the appropriate place.

**SALAI SANMARGAM J (810022104072)**

**SATHISH S (810022104075)**

I certify that the declaration made above by the candidate is true.

.

**Signature of the Guide**

**Dr. P. KARTHIKEYAN**
Assistant Professor,
Department of CSE
University College of Engineering,
BIT Campus,
Anna University,
Tiruchirappalli-620 024

# ACKNOWLEDGEMENT

We would like to thank our honorable Dean **Dr. T. SENTHILKUMAR**, **Professor**, for having provided us with all the required facilities to complete our project without hurdles.

We would also like to express our sincere thanks to **Dr. G. ANNAPOORANI**, **Assistant Professor (Sl. Gr), Head of the Department of CSE/IT**, for her valuable guidance, suggestions and constant encouragement paved the way for the successful completion of this project work.

We would like to thank our Project Coordinators **Dr. C. USHARANI**, **Assistant Professor**, Department of Computer Science and Engineering, for her kind support.

We would like to thank and express our deep sense of gratitude to our project guide **Dr. P. KARTHIKEYAN, Assistant Professor**, Department of Computer Science and Engineering, for her valuable guidance throughout the project. We also extend our thanks to all other teaching and non-teaching staff for their encouragement and support.

We thank our beloved parents and friends for their full support in the moral development of this project.

# TABLE OF CONTENTS

# ABSTRACT

Hospital Management Systems are safety-critical applications that demand continuous availability, strict data confidentiality, and rapid yet reliable delivery of new features and security patches. Traditional manual deployment processes are no longer sufficient for healthcare software; they are slow, error-prone, and incapable of enforcing the rigorous validation that patient-facing systems require. Continuous Integration and Continuous Deployment (CI/CD) pipelines have emerged as a way to automate software delivery, but they introduce a new class of risks at the input, runtime, and output stages, particularly at the output stage (AS3) where final artifacts are released. This project designs and implements a secure, automation-driven CI/CD pipeline for a hospital software system, using a modular Hospital Management System (HMS) as a real-world case study. The pipeline integrates secret scanning with Gitleaks, backend linting and unit testing and Pytest, frontend type-checking and bundling with TypeScript and Vite, container image publication to GitHub Container Registry, container vulnerability scanning with Trivy, and controlled deployment to Vercel with a managed PostgreSQL backend. Each stage acts as a mandatory quality gate; failure at any gate halts the pipeline and prevents insecure code from reaching production. The HMS application itself is a three-tier system built with a React 19 and TypeScript single-page frontend, a FastAPI 3.11 backend, and a PostgreSQL 16 database, supporting seven user roles and twelve operational modules including patient management, appointments, laboratory, pharmacy, ward and bed management, prescriptions, nurse orders, billing, and audit logging. The complete pipeline executes in five to ten minutes, reliably blocks unsafe deployments, and demonstrates that strong DevSecOps practices can be achieved with lightweight, accessible tooling. The project shows that automation, when combined with mandatory validation and security gates, can be used as a governance mechanism that improves software quality, reliability, and patient safety in healthcare environments.

# சுருக்கம்

நவீன சுகாதார சூழலில், மருத்துவமனை மேலாண்மை அமைப்புகள் (Hospital Management Systems - HMS) நோயாளர் பதிவுகள், நேர்முக சந்திப்புகள், கட்டணங்கள் மற்றும் மருத்துவ செயல்பாடுகளை நிர்வகிப்பதில் முக்கிய பங்கு வகிக்கின்றன. இந்த அமைப்புகள் தொடர்ச்சியான சேவை வழங்கல் மற்றும் நுணுக்கமான நோயாளர் தகவல்களை பாதுகாக்க உயர் நம்பகத்தன்மை, அடிக்கடி புதுப்பிப்புகள் மற்றும் வலுவான பாதுகாப்பு தேவையாகும். Continuous Integration மற்றும் Continuous Deployment (CI/CD) குழாய்கள் மென்பொருள் வெளியீட்டின் வேகத்தையும் ஒருமைப்பாட்டையும் அதிகரித்துள்ளன. ஆனால், பல CI/CD அமைப்புகளில் பாதுகாப்பு சரிபார்ப்பு போதுமானதாக இல்லாததால் சார்பு (dependency) குறைபாடுகள், கட்டமைப்பு பிழைகள் மற்றும் பாதுகாப்பற்ற பயன்பாடுகள் வெளியீடு போன்ற அபாயங்கள் உருவாகின்றன. இந்த திட்டம், மருத்துவமனை மென்பொருளுக்கான பாதுகாப்பான CI/CD குழாயை வடிவமைத்து செயல்படுத்துவதைக் குறிக்கோளாகக் கொண்டுள்ளது. இதற்காக Flask அடிப்படையிலான backend, தொடர்புடைய தரவுத்தளம் மற்றும் frontend உடன் HMS பயன்பாடு உருவாக்கப்பட்டுள்ளது. GitHub Actions பயன்படுத்தி CI/CD செயல்முறை தானியங்கமாக இயங்குகிறது. இதில் pytest மூலம் சோதனை, Docker மூலம் containerization மற்றும் Trivy மூலம் பாதுகாப்பு குறைபாடுகள் கண்டறிதல் போன்ற கட்டங்கள் உள்ளன. இந்த திட்டத்தின் முக்கிய பங்களிப்பு பாதுகாப்பு gate அமைப்பு ஆகும். இது முக்கியமான பாதுகாப்பு குறைபாடுகள் இருந்தால் deployment-ஐ தடுக்கிறது, இதனால் பாதுகாப்பான மென்பொருள் மட்டுமே வெளியிடப்படுகிறது. மேலும், logging மற்றும் monitoring அம்சங்கள் செயல்திறன் கண்காணிப்பையும் நம்பகத்தன்மையையும் மேம்படுத்துகின்றன. மொத்தத்தில், இந்த திட்டம் automation மற்றும் security இணைப்பின் மூலம் மென்பொருள் தரம் மற்றும் பாதுகாப்பை உயர்த்தும் ஒரு பயனுள்ள மற்றும் விரிவாக்கக்கூடிய முறையை வழங்குகிறது.

# LIST OF TABLES

# LIST OF FIGURES

# LIST OF ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| AS1 / AS2 / AS3 | Attack Surface 1 (Input), 2 (Runtime), 3 (Output) |
| CI / CD | Continuous Integration / Continuous Deployment |
| CVE | Common Vulnerabilities and Exposures |
| DFD | Data Flow Diagram |
| DevSecOps | Development, Security, and Operations |
| FHIR | Fast Healthcare Interoperability Resources |
| GHCR | GitHub Container Registry |
| HMS | Hospital Management System |
| HL7 | Health Level Seven |
| JWT | JSON Web Token |
| RBAC | Role-Based Access Control |
| SARIF | Static Analysis Results Interchange Format |
| SBOM | Software Bill of Materials |
| SDLC | Software Development Life Cycle |
| SPA | Single Page Application |
| SSL / TLS | Secure Sockets Layer / Transport Layer Security |

# CHAPTER 1

# INTRODUCTION

## 1.1 Background of the Study

The rapid advancement of information technology has significantly transformed the way hospital software systems are developed and deployed. Modern Hospital Management Systems (HMS) are required to operate continuously, handle sensitive patient data, and support critical healthcare operations such as appointments, diagnosis records, and billing. These systems must be updated frequently to improve functionality, fix errors, and address security vulnerabilities while ensuring uninterrupted service. Traditional software deployment methods rely on manual processes that are not suitable for hospital environments. Manual deployment increases the risk of human errors, inconsistent configurations, and system failures, which can directly impact patient care and hospital operations. To overcome these challenges, Continuous Integration and Continuous Deployment (CI/CD) pipelines are used to automate software development and deployment processes.CI/CD pipelines enable automated code integration, testing, and deployment, ensuring faster and more reliable software delivery. In hospital software systems this automation is essential to maintain system stability and reduce downtime. However, CI/CD pipelines also introduce security risks if not properly designed, such as deploying vulnerable software or misusing system privileges. This project focuses on implementing a secure and automated CI/CD pipeline for hospital software systems. The proposed approach integrates automated testing, containerization, and vulnerability scanning to ensure that only validated and secure software is deployed. A Hospital Management System is used as a case study to demonstrate how automation-driven pipelines can improve reliability, reduce human errors, and enhance security in healthcare applications.

Continuous Integration ensures that code changes are frequently merged into a shared repository where automated tests validate functionality. Continuous Deployment extends this process by automatically delivering validated updates without service interruption. Together, these practices enforce consistency across development, testing, and production environments. When combined with containerization and security scanning, CI/CD becomes more than an automation convenience; it becomes a governance mechanism that systematically prevents unsafe code from reaching production. In this project, Docker-based containerization is used to package the Hospital Management System into a consistent and portable runtime environment.Automated tests are integrated into the pipeline to verify core functions such as authentication, patient records, appointments, and billing before deployment.Security scanning tools are included to detect vulnerabilities early and prevent unsafe images or dependencies from reaching production. This approach improves deployment reliability, supports faster updates, and ensures the hospital system remains stable, secure, and available for continuous use.

## 1.2 Problem Statement

Modern hospital software systems must support continuous updates, real-time operations, and high availability while maintaining strict standards of reliability and security. Applications such as HMS handle critical tasks including patient data management, appointment scheduling, diagnostics, and billing. Healthcare organizations increasingly adopt CI/CD pipelines to automate software build, test, and deployment processes; however, when not properly designed these pipelines introduce significant challenges for system reliability, security enforcement, and operational governance.

In many hospital software environments CI/CD pipelines are only partially automated, focusing mainly on basic build and test stages. Critical validation steps such as artifact verification, security scanning, and deployment control are often treated as optional or performed manually. Deployment decisions may be influenced by urgency in resolving operational issues rather than by strict validation outcomes. As a result, insecure or unstable software artifacts can reach live hospital systems without adequate verification, increasing the risk of system failures, inconsistent data handling, and exposure of sensitive patient information.

A particular concern is the vulnerability of the output stage (AS3), where final application artifacts such as Docker images are released for deployment. These artifacts are typically assumed to be trusted and are deployed directly into production. Without proper validation, attackers can exploit outdated dependencies, insecure configurations, or pipeline misconfigurations to inject malicious code into these artifacts. Once deployed, compromised artifacts can lead to data breaches, service interruptions, unauthorized access to medical records, and disruption of clinical workflows. There is therefore a strong need for a practical, automation-driven CI/CD pipeline tailored for hospital software systems that enforces mandatory validation and security checks at every stage and is lightweight enough for academic and small-team environments.

## 1.3 Objectives of the Project

The primary objective of this project is to design and implement a secure and automated CI/CD pipeline for hospital software systems, ensuring reliable, consistent, and safe deployment of applications such as Hospital Management Systems. In healthcare environments where system failures can directly affect patient care, it is essential to maintain high levels of reliability and security in software delivery.

The specific objectives of the project are:

- To automate the key stages of the software development life cycle including code integration, automated testing, containerization, and deployment, eliminating manual intervention.
- To use Docker containerization so that the hospital application runs identically across development, testing, and production environments.

- To integrate automated vulnerability scanning into the pipeline so that insecure dependencies and container images are detected and blocked before deployment.
- To establish a security gate at the output stage (AS3) that prevents deployment whenever critical vulnerabilities are present.
- To provide monitoring, logging, and audit capabilities that allow the team to track pipeline execution and respond quickly to issues.
- To validate the design end-to-end through a real Hospital Management System case study covering seven user roles and twelve operational modules.

## 1.4 Scope of the Project

The scope of this project is centered on designing and implementing a secure and automated CI/CD pipeline for hospital software systems, with a focus on the reliable and safe deployment of applications such as a Hospital Management System. The project addresses the risks of deploying untested or insecure software in healthcare environments using lightweight, accessible tooling rather than complex enterprise infrastructure.

The current phase of HMS targets the operational core of a small to medium hospital. The following capabilities are within scope and are implemented in the project as submitted:

- User registration and login with role assignment, including a patient self-registration path that creates both a user account and a linked clinical patient record.
- Patient lifecycle management covering registration, demographic and clinical metadata, and status transitions between outpatient, inpatient, and discharged states.
- Appointment scheduling and lifecycle management with role-filtered visibility for doctors and patients and broader visibility for receptionists and administrators.
- Vitals capture and patient-level vitals retrieval, including the latest record per patient and historical lookups.
- Laboratory workflow including test ordering by doctors, status updates by lab technicians, department-based routing, and result capture.
- Pharmacy inventory management with stock level tracking, threshold-based stock status, and prescription-linked dispensing.
- Ward and bed management with availability tracking, patient assignment to beds, and admission and discharge timestamps.
- Billing pipeline with module-sourced billing contributions, finalized bills, payment entries, balance computation, and consolidated patient financial state.
- Continuous integration workflows for backend and frontend, secret scanning, container image publication, image vulnerability scanning, and a deployment workflow.

## 1.5 Motivation

The motivation for this project arises from both academic research and real-world challenges in modern software development. Studies show that many CI/CD pipelines operate with outdated components, insufficient validation, and weak security controls, leading to deployment failures, vulnerability exposure, and system instability. In many cases these issues are not caused by complex cyber-attacks but by human errors such as skipping tests, misconfigurations, or insecure dependencies.

In hospital systems these risks become significantly more critical. HMS platforms hold sensitive patient data and support core healthcare operations; any failure in software deployment can disrupt clinical services and directly affect patient care. CI/CD provides automation to improve deployment speed and consistency, but automation alone is insufficient without proper validation and security enforcement. There is a strong need for disciplined CI/CD pipelines that ensure all updates are thoroughly tested, validated, and secured before deployment, using lightweight tools accessible to academic projects and small healthcare IT teams.

## 1.6 Significance of the Study

This project is significant for several reasons. First, it demonstrates that strong DevSecOps practices can be achieved with widely available tools such as GitHub Actions, Docker, Trivy, and Vercel, without requiring large-scale enterprise infrastructure. Second, it positions automation not merely as a convenience but as a governance mechanism: pipelines act as policy enforcement points that decide whether code is allowed to reach production. Third, the project specifically targets the output stage of the pipeline, which research has identified as a major attack surface where outdated dependencies and tampered artifacts cause real-world incidents.

Finally, the study contributes a documented end-to-end design and implementation that bridges theoretical research and practical engineering. It can serve as a reference for students, small healthcare IT teams, and similar safety-critical domains where reliability, traceability, and the prevention of insecure releases are non-negotiable requirements.

# CHAPTER 2

# LITERATURE REVIEW

## 2.1 Introduction

The rapid evolution of software development practices has led to widespread adoption of CI/CD pipelines as a fundamental component of modern DevOps environments. These pipelines enable automated integration, testing, and deployment of software, improving development efficiency, reducing time-to-market, and ensuring consistent delivery of applications. However, as CI/CD pipelines become more complex and widely used, they also introduce new operational and security challenges that must be carefully addressed.

A literature review plays a crucial role in understanding the current state of research in this domain. It provides a structured analysis of existing studies related to CI/CD automation, security risks, DevSecOps practices, and mitigation strategies. By examining prior work it becomes possible to identify key trends, challenges, and limitations in current approaches, as well as opportunities for improvement. In the context of this project the review focuses particularly on security vulnerabilities within CI/CD pipelines, with special emphasis on the output stage where final software artifacts are generated and deployed.

Recent research highlights that while CI/CD pipelines improve productivity, they often lack proper governance, validation, and security enforcement. Studies such as Pan et al. (2024) reveal that a large percentage of pipelines operate with outdated scripts and configurations, making them vulnerable to credential leakage, build environment compromise, and malicious artifact injection. DevSecOps-focused research emphasizes integrating security mechanisms directly into the pipeline rather than treating security as a separate or post-deployment activity.

## 2.2 Review of Existing Systems and Research Papers

### 2.2.1 Paper 1 — Pan et al. (2024)

Pan and colleagues conducted a large-scale empirical study of over 320,000 GitHub repositories to analyze security threats in CI/CD pipelines. The study identified three major attack surfaces — Input (AS1), Runtime (AS2), and Output (AS3). It revealed that 97.86% of repositories rely on outdated scripts, resulting in an average security update delay of approximately 11 months. The research further explains how attackers exploit these weaknesses to leak sensitive credentials, gain unauthorized root access, and inject malicious backdoors into final build artifacts. The findings emphasize the importance of timely updates, secure configurations, and reduced privilege access in automated pipelines.

***Limitations:*** *Focuses primarily on identifying vulnerabilities but does not provide a simplified or practical implementation strategy suitable for small-scale or academic projects.*

### 2.2.2 Paper 2 — Ashtagi et al. (2025)

Ashtagi et al. proposed a security-first DevSecOps framework designed to build resilient CI/CD pipelines. The framework integrates Jenkins for continuous integration, ArgoCD for GitOps-based deployment, and Terraform for Infrastructure as Code. It incorporates automated vulnerability scanning using OWASP Dependency Check and Trivy, achieving a reduction of over 70% in initial vulnerabilities. Additionally, observability tools such as Prometheus, Grafana, and Loki were used to monitor system performance and detect anomalies in real time. This approach demonstrates how integrating security into every stage of the pipeline enhances reliability and reduces manual intervention.

***Limitations:*** *The framework depends heavily on cloud infrastructure and advanced tools, making it complex and resource-intensive for beginners or small-scale implementations.*

### 2.2.3 Paper 3 — Shahin et al. (2017)

Shahin et al. presented a comprehensive systematic review of Continuous Integration, Delivery, and Deployment practices. The study categorizes various tools, approaches, and challenges involved in CI/CD adoption. It highlights that automation of build, testing, and deployment processes significantly improves software quality and reduces time-to-market. However, the study also identifies challenges such as maintaining automated test environments and ensuring infrastructure readiness. The research is particularly relevant for critical systems, where maintaining stability and data integrity is essential.

***Limitations:*** *Provides theoretical insights but lacks practical implementation details or case studies specific to healthcare or real-time systems.*

### 2.2.4 Paper 4 — Mowad et al. (2022)

Mowad et al. examined how CI/CD practices help bridge the gap between development and operations teams in a DevOps environment. Through case studies, the authors demonstrate that continuous practices enable teams to manage the entire software lifecycle efficiently, reducing integration issues and improving release stability. The study highlights the role of cloud platforms

such as Azure DevOps in enabling seamless and automated deployments. It shows that automation significantly minimizes human errors and enhances collaboration across teams.

***Limitations**: Focuses more on organizational and workflow improvements rather than addressing detailed security challenges within CI/CD pipelines.*

### 2.2.5 Paper 5 — Faqih et al. (2024)

Faqih et al. conducted an empirical analysis of GitHub Actions workflows to study the usage of CI/CD tools. The research categorizes tools into five domains: version control, static analysis, build automation, test automation, and CI/CD orchestration. It concludes that build automation is the most widely used practice and highlights the strong relationship between programming languages and tool selection. The study emphasizes the importance of automated testing and validation, especially in containerized environments, to ensure reliable software delivery.

***Limitations:** The study is observational and does not propose new techniques or improvements for enhancing CI/CD pipeline security or performance.*

### 2.2.6 Paper 6 — Gusmedi et al. (2025)

Gusmedi et al. presented a case study on improving digital service reliability in a hospital system using DevOps practices. The system integrates auto-scaling and load balancing to manage varying workloads efficiently while ensuring high availability. The study highlights how infrastructure optimization techniques can prevent downtime and maintain continuous access to healthcare services. It demonstrates that reliability is a critical requirement in healthcare environments where system failures can directly impact patient care.

***Limitations:** Focuses mainly on infrastructure reliability and scalability, with limited emphasis on CI/CD pipeline security and software-level automation.*

### 2.2.7 Paper 7 — Mohammed et al. (2025)

Mohammed et al. explored the evolution of DevSecOps and its impact on application security through a systematic literature review. The study emphasizes the concept of “Shift-Left” security, where security testing is integrated early in the development lifecycle. It highlights the importance of automated static code analysis and container scanning to identify vulnerabilities before deployment. The research establishes that embedding security into CI/CD pipelines significantly reduces risks and improves overall system security.

***Limitations:*** *Provides conceptual and theoretical insights without demonstrating a concrete implementation model or real-world deployment scenario.*

### 2.2.8 Paper 8 — Tøndel et al. (2022)

Tøndel et al. investigated how security prioritization is managed in agile software development environments. The study reveals that security is often deprioritized in favor of functional requirements due to time constraints. It suggests that integrating automated security tools and involving security experts within development teams can improve security awareness and practices. The research highlights the importance of aligning security with development workflows to ensure consistent protection.

***Limitations:*** *Focuses on team behavior and management strategies rather than providing technical solutions for securing CI/CD pipelines.*

### 2.2.9 Paper 9 — Muñoz et al. (2021)

Muñoz et al. proposed the P2ISE system to preserve the integrity of CI/CD pipelines using secure hardware elements. The system introduces a "Root of Trust" mechanism to verify and sign every stage of the pipeline, ensuring that build artifacts are not tampered with. This approach effectively protects against unauthorized changes, malicious code injection, and supply chain attacks. The study highlights the importance of artifact integrity in automated software delivery systems.

***Limitations:*** *The use of hardware-based security mechanisms increases implementation complexity and may not be feasible for low-budget or academic projects.*

### 2.2.10 Paper 10 — Mooduto and Rijanto (2023)

Mooduto and Rijanto proposed an optimized CI/CD pipeline integrating OWASP Dependency Track with AWS CodePipeline for efficient vulnerability management. The system automates Software Bill of Materials (SBOM) generation and detects vulnerable dependencies in real time. Their approach significantly reduces execution time to under four minutes while maintaining strong security checks. The study demonstrates that security automation can be both efficient and scalable without affecting deployment speed.

***Limitations:*** *The approach is tightly coupled with AWS services, limiting flexibility and portability to other platforms or local environments.*

## 2.3 Output Stage (AS3) Vulnerabilities

The output stage (AS3) of a CI/CD pipeline is a critical point where final software artifacts, such as Docker images or application builds, are prepared for deployment. In hospital software systems this stage is highly sensitive because these artifacts directly affect patient management, medical records, and clinical operations.

Research shows that attackers can exploit weaknesses in this stage without modifying source code by injecting malicious code into build artifacts. If outdated dependencies or insecure configurations are used, vulnerable software may be deployed into hospital systems, leading to data breaches or service disruptions. Another major risk is the lack of artifact validation: because output artifacts are trusted by default, any compromise at this stage spreads across all deployment environments. In hospital systems this can affect multiple services simultaneously, multiplying the impact of an attack.

To address these risks, automated validation techniques such as vulnerability scanning and artifact integrity checks are essential. By enforcing security checks before deployment, hospital software pipelines can ensure that only secure and verified applications are released. Securing the output stage is therefore crucial for protecting hospital systems and ensuring safe and reliable healthcare operations.

## 2.4 Existing Approaches for Risk Mitigation

The literature describes several approaches for mitigating risks in CI/CD pipelines. Static and dynamic analysis tools detect insecure code patterns before deployment. Software composition analysis identifies vulnerable third-party libraries. Container image scanners such as Trivy, Grype, and Anchore inspect operating-system packages and language libraries inside images. Secret scanners such as Gitleaks and TruffleHog detect hardcoded credentials in repositories. Policy-as-code tools such as Open Policy Agent and Conftest enforce organizational rules in pipelines.

More advanced approaches include hardware-rooted trust models that sign and verify artifacts cryptographically, as well as Software Bill of Materials (SBOM) generation and continuous dependency tracking. Despite their effectiveness, many of these techniques rely on complex cloud-native infrastructures, paid services, or specialized hardware, putting them out of reach for academic projects and small healthcare IT teams. The proposed system in this project applies the most impactful techniques — secret scanning, automated testing, container vulnerability scanning, and gated deployment — using free and widely available tools.

## 2.5 Comparative Analysis of Existing Systems

Table 2.1 compares the surveyed studies on year, key findings, advantages, and limitations. The synthesis confirms a clear gap: most existing approaches either focus narrowly on identification

of vulnerabilities or assume large-scale enterprise infrastructure. A practical, lightweight, security-enforced pipeline tailored for hospital software remains underexplored.

*Table 1: Comparative Analysis of Existing Systems*

| Year | Author(s) | Key Findings | Advantages | Limitations |
|---|---|---|---|---|
| 2024 | Pan et al. | Identified AS1/AS2/AS3 attack surfaces; 97.86% repos use outdated scripts; ~11 months update lag. | Large-scale empirical study (320k repos); clearly defines CI/CD attack model. | No lightweight practical implementation. |
| 2025 | Ashtagi et al. | DevSecOps framework with OWASP+Trivy on AWS; 70% vuln reduction. | Effective automated security integration; cloud-native. | Requires complex infrastructure; enterprise setup. |
| 2017 | Shahin et al. | Systematic review of CI/CD; automation improves quality and time-to-market. | Comprehensive academic foundation. | Older study; limited container security focus. |
| 2022 | Mowad et al. | CI/CD bridges Dev–Ops gap; better lifecycle ownership. | Operational efficiency; reduced integration issues. | Less focus on security enforcement. |
| 2024 | Faqih et al. | GitHub Actions workflows emphasize build & test automation. | Industry-aligned empirical data. | Limited security-depth analysis. |
| 2025 | Gusmedi et al. | DevOps with auto-scaling improves hospital system reliability. | Healthcare-specific case study. | Focuses on infrastructure scaling. |
| 2025 | Mohammed et al. | Shift-Left security essential in DevSecOps. | Strong support for automated security gates. | Conceptual review; lacks beginner-level case. |
| 2025 | Williams et al. | Compared 9 GitHub security scanners. | Practical scanner comparison. | Detection varies; no single tool covers all. |

| Year | Author(s) | Key Findings | Advantages | Limitations |
|---|---|---|---|---|
| 2022 | Tøndel et al. | Security often deprioritized in Agile. | Behavioural & governance perspective. | Limited technical pipeline detail. |
| 2021 | Muñoz et al. | P2ISE for artifact integrity; Root of Trust. | Strong AS3 mitigation. | Hardware secure elements; complex setup. |
| 2023 | Mooduto & Rijanto | SBOM & dependency tracking via AWS CodePipeline. | Efficient automation (<4 min). | AWS-dependent. |
| 2024 | Hyun et al. | Jenkins automation improves deployment consistency. | Statistical proof of automation reliability. | Limited deep security integration. |
| 2024 | Rajendra et al. | Cloud-based CI/CD for web deployment. | Practical workflow; high availability. | Less focus on output-stage scanning. |
| 2023 | Wróbel et al. | CI maturity drives reliability; failure handling matters. | Emphasizes failure protocol. | Limited supply chain focus. |
| 2025 | Ramos & Yoo | Major threats: secret exposure, uncontrolled automation. | Comprehensive systematic review. | Theoretical; no small-team solution. |

# CHAPTER 3

# PROPOSED SYSTEM

## 3.1 Problem Definition

The problem addressed by this project is the absence of a practical, security-enforced CI/CD pipeline tailored for hospital software systems. Existing manual or partially automated deployment workflows in healthcare are slow, inconsistent, and error-prone, and they routinely allow unverified or vulnerable artifacts to reach production. The output stage in particular is poorly governed, leaving hospitals exposed to outdated dependencies and tampered images. The project must therefore design an end-to-end pipeline that automates build, test, security scanning, and deployment for an HMS application using lightweight, accessible tools, while making each stage a mandatory quality and security gate.

## 3.2 Existing System

In traditional hospital software environments, application deployment is largely performed using manual or partially automated processes. Developers implement updates such as new features, bug fixes, or system improvements, test them locally or in limited environments, and then manually deploy these updates to hospital servers or production systems. While this approach may be manageable for smaller applications with infrequent updates, it becomes inefficient, time-consuming, and highly error-prone as systems grow in complexity and require continuous updates. Even in healthcare organizations that adopt basic CI/CD practices, automation is often limited to simple tasks such as code compilation and basic testing. Many pipelines automatically trigger builds when code is updated, yet critical stages such as security validation, artifact verification, and controlled deployment are missing or handled manually. As a result, these pipelines appear automated but still depend heavily on human intervention. Deployment decisions are often influenced by operational urgency, which leads to skipped tests and rushed releases. Different developers or system administrators may follow different procedures when deploying software, producing inconsistent behavior across development, testing, and production.

### 3.2.1 Limitations of Existing System

Despite being widely used, existing hospital software deployment systems exhibit several critical limitations that affect reliability, security, and overall suitability for healthcare environments:

- **High dependency on manual intervention.** Repeated manual errors during deployment cause application failures, incorrect data processing, and downtime.

- **Lack of enforced security validation.** Vulnerability scanning, dependency checking, and artifact verification are often optional or done after deployment.
- **Uncontrolled output artifacts (AS3 vulnerability).** Final builds and Docker images are trusted without verification, allowing vulnerable or outdated components into production.
- **Inconsistent deployment environments.** Differences in configurations, libraries, and runtimes cause unexpected behavior in clinical settings.
- **Poor traceability and monitoring.** Insufficient logging makes troubleshooting and audit compliance difficult.
- **Not suitable for safety-critical applications.** These limitations together make existing approaches inadequate for systems where downtime or insecure releases impact patient care.

## 3.3 Proposed System Overview

The proposed system introduces a fully automated, security-enforced CI/CD pipeline specifically designed for hospital software systems such as HMS. It addresses the limitations of existing deployment practices by integrating automation not only for execution but also as a governance and enforcement mechanism. Every software update strictly follows predefined validation and security rules before deployment.

At the core of the system is a centralized version control repository acting as the single source of truth. Each code change automatically triggers the pipeline. The pipeline is divided into sequential stages, each acting as a mandatory checkpoint: code integration and build, automated testing, Docker-based containerization, vulnerability scanning at the output stage, and controlled deployment. If any stage fails — through failing tests or detected vulnerabilities — the pipeline stops and prevents deployment. The Hospital Management System is used as the case study to demonstrate the design end-to-end.

### 3.3.1 Advantages of Proposed System

- **Automation-driven risk reduction.** Eliminates manual intervention; every update follows the same workflow, lowering human error.
- **Enforced output-stage security (AS3 mitigation).** Vulnerability scanning ensures only safe Docker images are deployed.
- **Consistent and reproducible environments.** Containerization removes drift between development, testing, and production.
- **Early detection of issues.** Automated testing and scanning catch problems before they reach production.
- **Improved traceability and auditability.** Detailed pipeline logs support compliance and post-incident investigation.

- **Scalability and adaptability.** The pipeline is modular and easy to extend with additional tools and environments.
- **Suitability for healthcare applications.** High reliability, data integrity, and secure deployment match HMS requirements.
- **Beginner-friendly implementation.** Uses GitHub Actions, Docker, and Trivy - free, widely used, and well documented.

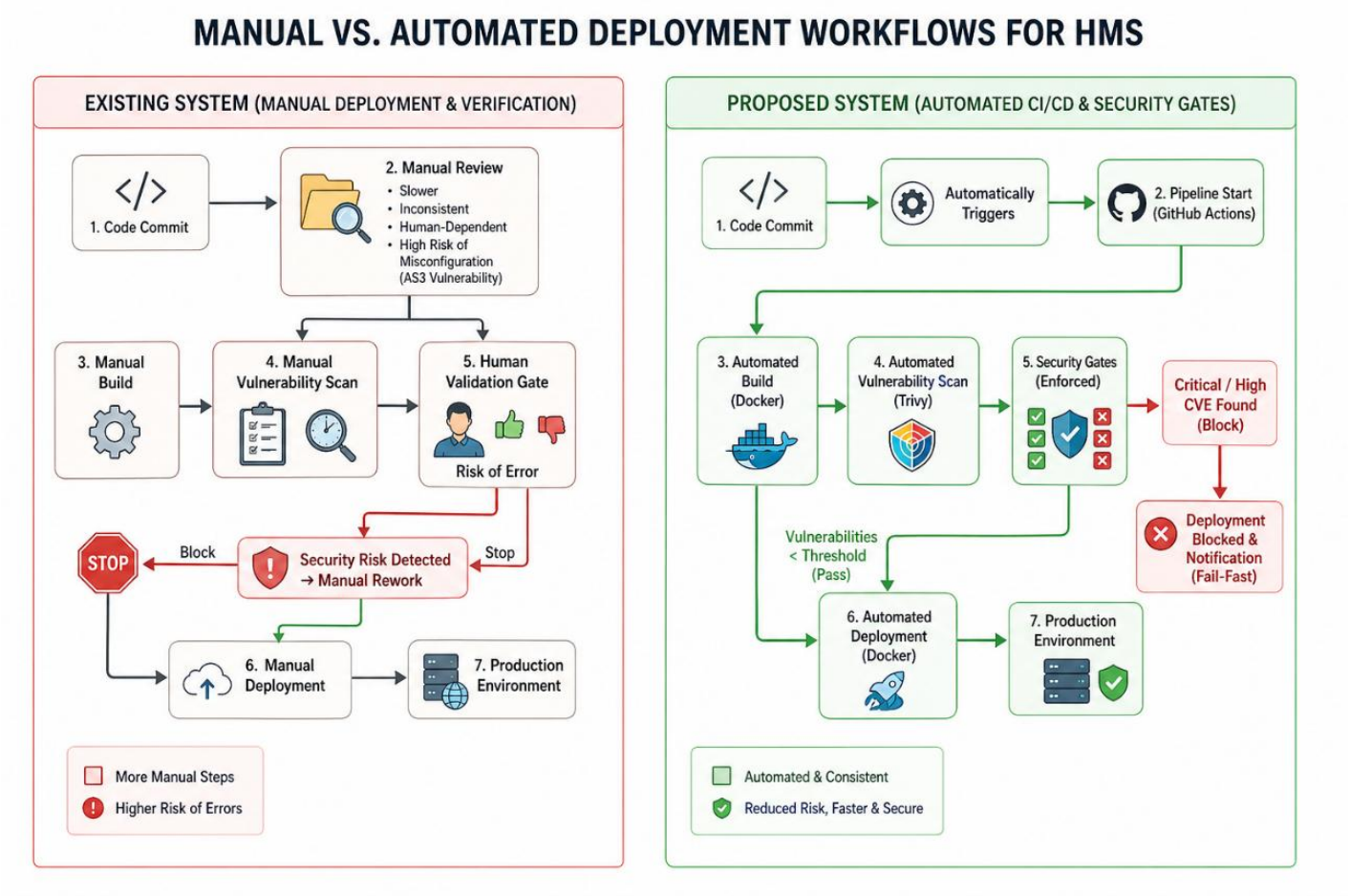



*Figure 1: Comparison of existing (manual) vs. proposed (automated) systems*

## 3.4 System Architecture

Architecture integrates development, automation, security, and deployment into a unified framework. Developers push updates to a centralized GitHub repository which acts as the single source of truth. Each push triggers the CI/CD automation layer, which performs code checkout, dependency installation, automated testing, build, and validation. A dedicated security layer scans dependencies and Docker images using Trivy. The deployment layer uses Docker containerization to release the application to its target environment, while a monitoring layer tracks pipeline execution and emits alerts. The result is a secure, automated, and reliable workflow for hospital software.

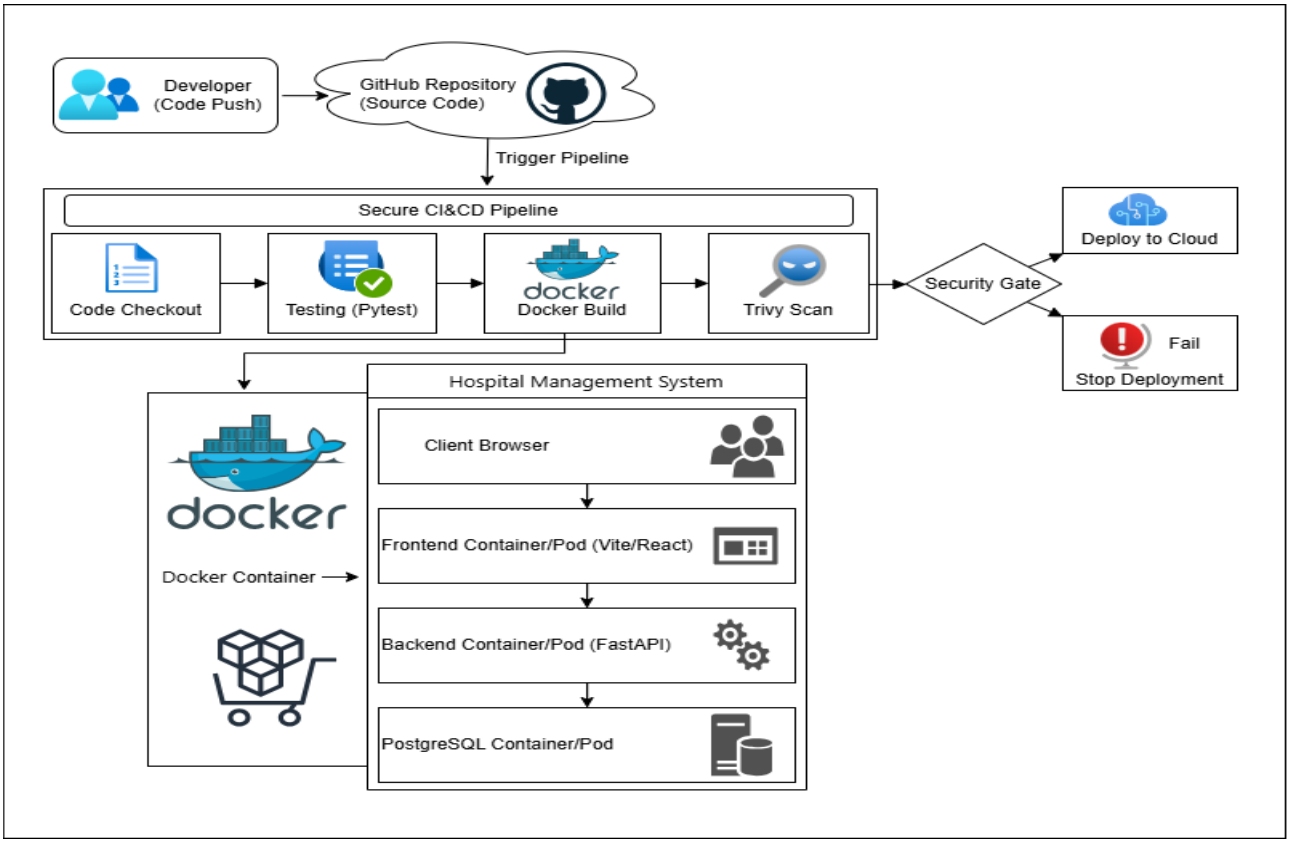



*Figure 2: System Architecture*

The system is realized through six GitHub Actions workflows that together form the pipeline:

*Table 2: GitHub Actions Workflow Files*

| Workflow File | Trigger | Purpose |
|---|---|---|
| ci-backend.yml | Push/PR (all branches) | Python linting (Ruff) + Pytest + environment verification |
| ci-frontend.yml | Push/PR (all branches) | TypeScript type-check + Vite build + artifact upload |
| ci-secret-scan.yml | Push/PR (all branches) | Gitleaks scan + .env tracking check + pattern scan |
| docker-publish.yml | Called by deploy | Build & push Docker images to GHCR |
| docker-scan.yml | Called by deploy | Trivy vulnerability scan on published images |
| deploy.yml | Push to main / staging | Orchestrates all gates → deploys to Vercel |

### 3.4.1 CI/CD Pipeline Architecture

The CI/CD pipeline architecture defines a structured workflow through which code moves from development to deployment. It is a sequence of automated stages, each acting as a quality gate. The pipeline is triggered when a developer pushes code to GitHub. The first stage is code checkout. Pytest then validates correctness. Once testing passes, the pipeline proceeds to the build stage where a Docker image is created. The next stage is vulnerability scanning, which is critical for security: Trivy scans the image for known CVEs, outdated dependencies, and misconfigurations. If critical issues are detected the pipeline stops immediately. If all stages pass, the pipeline proceeds to controlled deployment to the target environment. The architecture provides automated execution, early error detection, security enforcement, and controlled, reliable deployment.

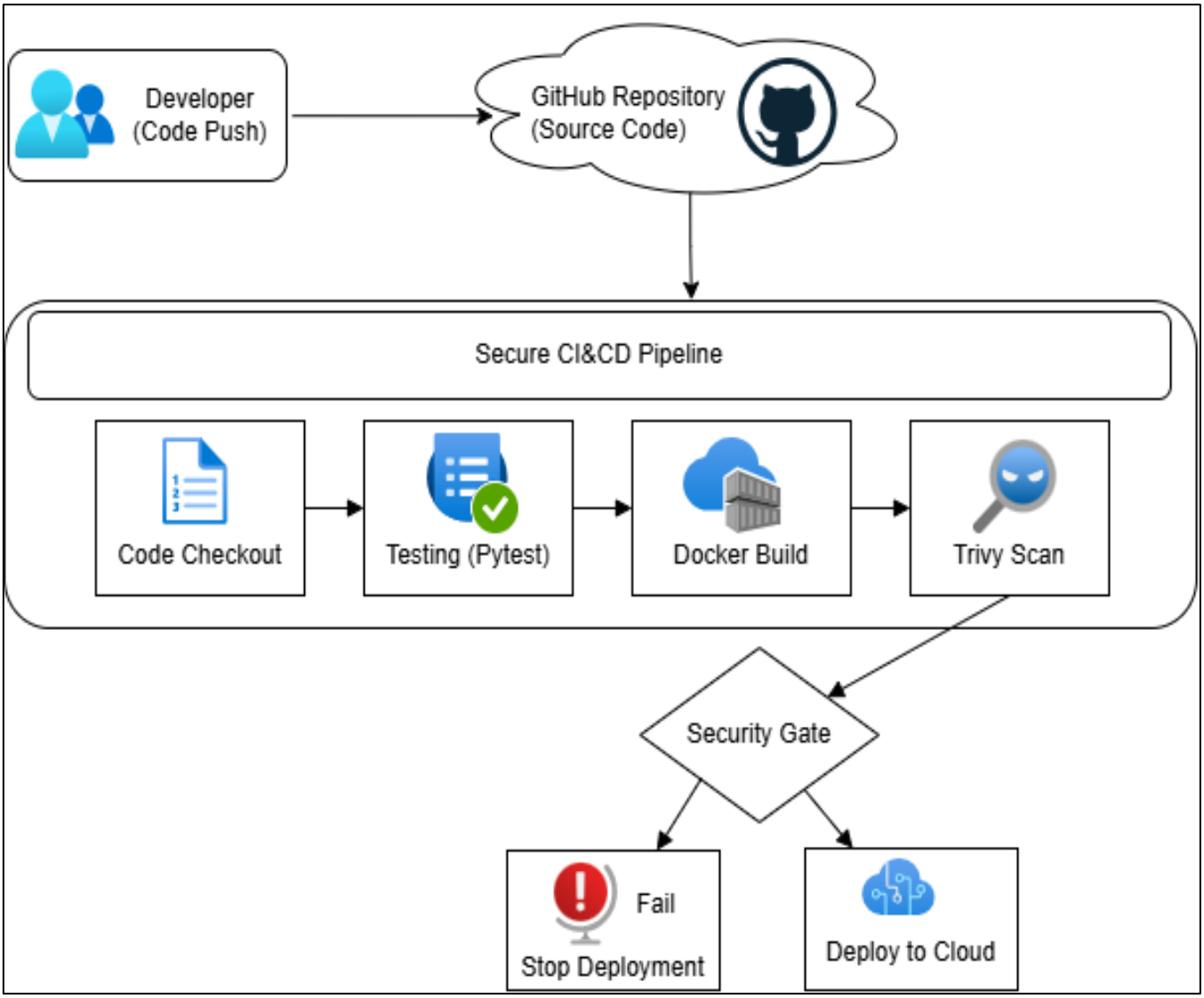



*Figure 3: CI/CD Pipeline Flow*

### 3.4.2 Application Architecture (3-Tier HMS)

The Hospital Management System uses a three-tier architecture that separates concerns and improves maintainability.

Tier 1 - the presentation tier is a Single Page Application built with React 19, TypeScript, and Vite. It runs in the user's browser and communicates with the backend through RESTful API calls over HTTPS.

Tier 2 - the application tier is a FastAPI backend on Python 3.11 served by Uvicorn. It contains business logic, authentication and authorization, validation, PDF invoice generation, and the REST API endpoints; it accesses the database through psycopg2's ThreadedConnectionPool.

Tier 3 - the data tier is a PostgreSQL 16 relational database. Locally it runs as the postgres:16-alpine container; in production it uses Neon DB, a serverless PostgreSQL service offering automatic scaling and zero-downtime upgrades. The schema contains 19 interrelated tables covering all hospital domains.

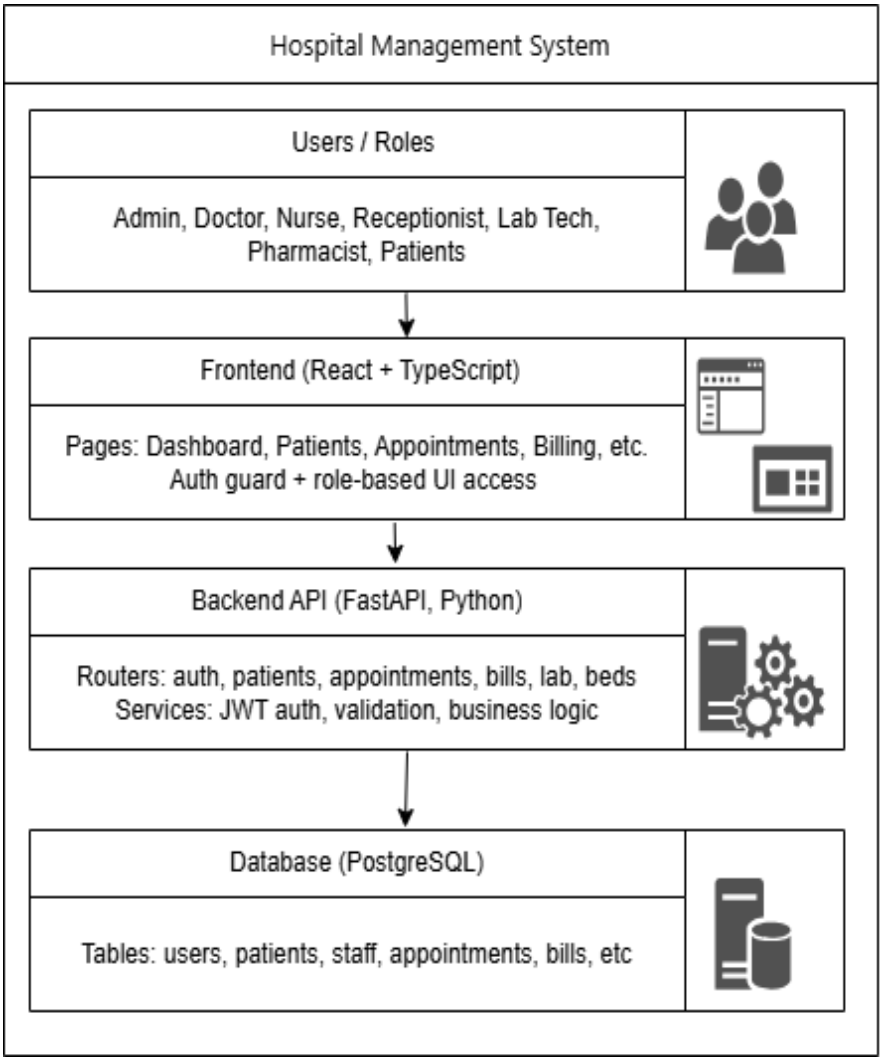



*Figure 4: HMS 3-Tier Architecture*

Communication between the tiers follows the protocols summarized in Table 3.

*Table 3: Communication Paths in the 3-Tier Architecture*

| Communication Path | Protocol | Format | Security |
|---|---|---|---|
| Browser ↔ Frontend | HTTPS | HTML / JS / CSS | TLS encryption |
| Frontend ↔ Backend | HTTPS REST | JSON | JWT Bearer tokens |
| Backend ↔ Database | TCP (PostgreSQL wire) | SQL | SSL/TLS (Neon) |

## 3.5 Data Flow Diagram (DFD)

### 3.5.1 Level 0 DFD

At the highest level the system has three main entities: the developer, the CI/CD pipeline, and the deployment environment. The developer submits code to the GitHub repository; the pipeline processes the code through testing, building, and security validation; and the validated application is deployed to the deployment environment.

### 3.5.2 Level 1 DFD

At a more detailed level the process includes:

- Code submission and storage in the repository.
- Automated testing and validation.
- Docker image creation.
- Vulnerability scanning.
- Deployment decision (pass / fail).
- Final deployment.

Data stores include the source code repository, Docker image artifacts, and the application database. Data flows in a controlled and automated manner so that only validated, secure artifacts ever reach production.

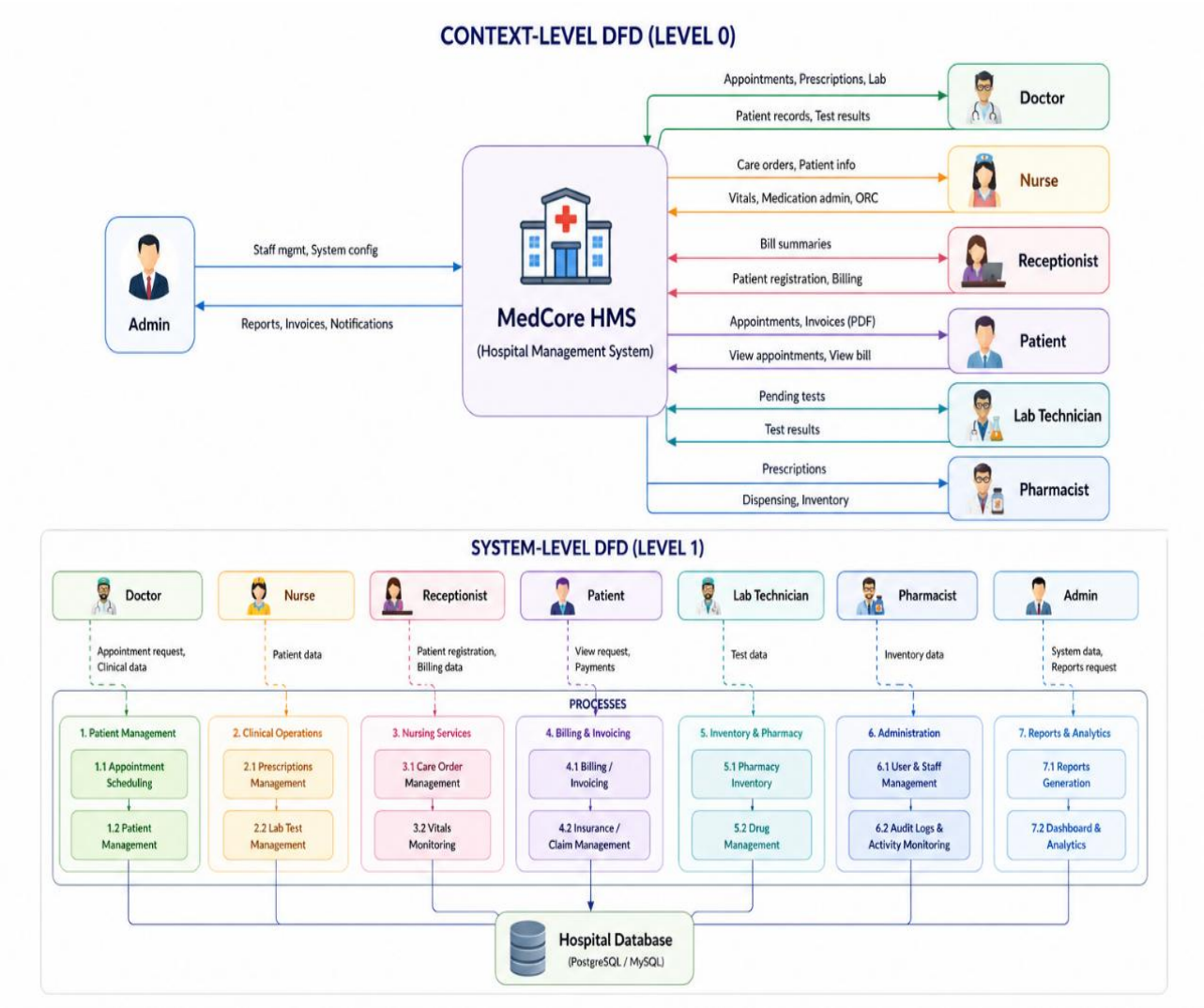



*Figure 5: Level 0 and Level 1 Data Flow Diagram*

## 3.6 Methodology

### 3.6.1 Software Development Life Cycle (SDLC)

The project followed a modified SDLC integrated with Agile sprints across six phases:

**Phase 1 - Requirements gathering and analysis.** Workflows of existing hospitals and prior HMS solutions were analyzed; functional and non-functional requirements were captured as user stories in a product backlog.

**Phase 2 - System design and architecture.** Three-tier architecture, 19-table database schema, REST contracts for thirteen routers, and a role-aware frontend component hierarchy were designed.

**Phase 3 - Implementation and coding.** FastAPI backend with parameterized SQL, connection pooling, and Pydantic validation; React 19 + TypeScript frontend with a custom CSS design system.

**Phase 4 - Testing and quality assurance.** Unit tests with Pytest, type-checking with the TypeScript compiler, Ruff linting, Gitleaks and Trivy security scans, and CI smoke tests.

**Phase 5 - Deployment and release.** Containerized with Docker, orchestrated locally with Docker Compose, and released to Vercel with Neon DB through a six-gate CI/CD pipeline.

**Phase 6 - Maintenance and iteration.** Issues and feature requests flow into the backlog; the audit log captures user actions for operational monitoring.

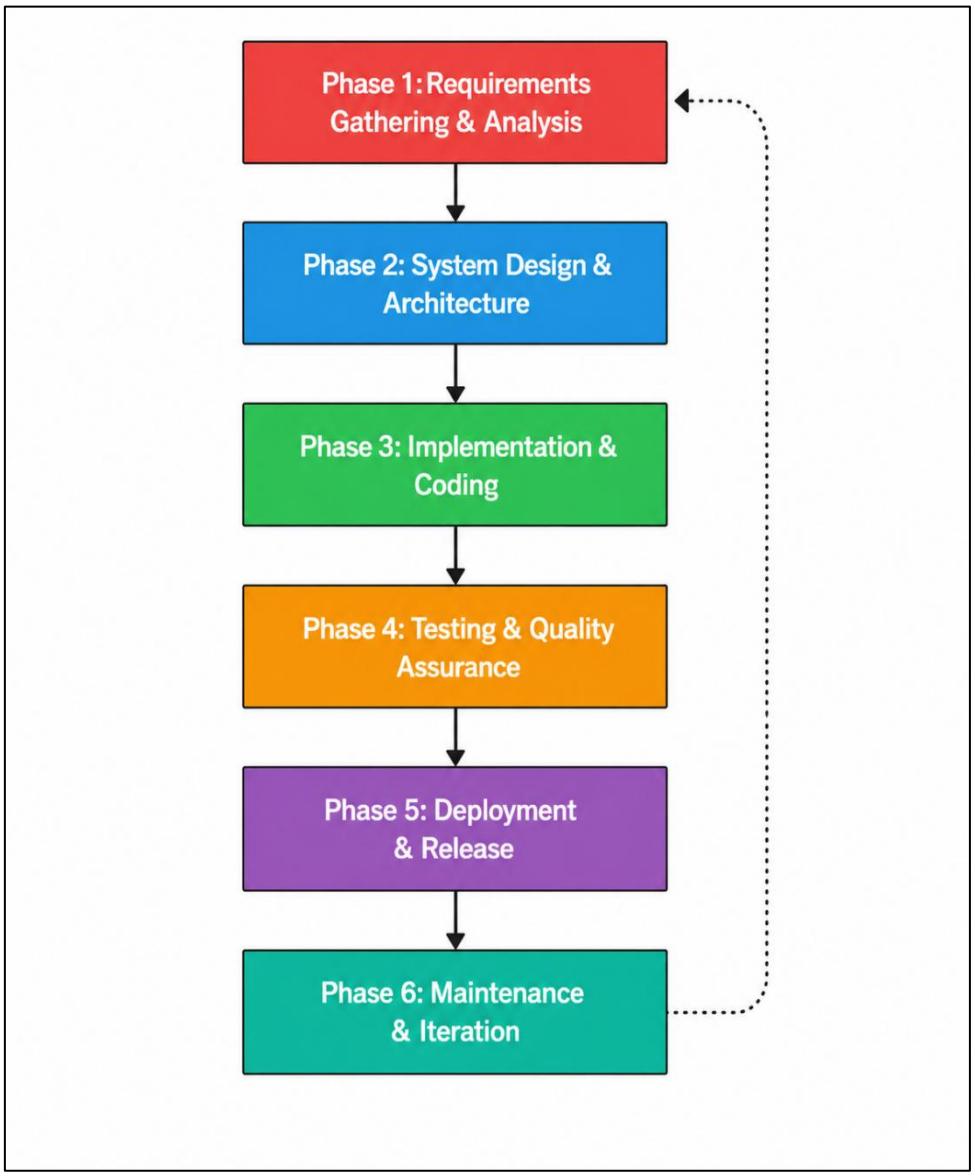



*Figure 6: Software Development Life Cycle Phases*

### 3.6.2 DevOps Practices

DevOps practices were integrated throughout the lifecycle to bridge development and operations: feature-branch workflows with code reviews, infrastructure-as-code through Docker Compose and Kubernetes manifests, continuous integration on every push, container image publication, pre-deployment vulnerability scanning, automated deployment to staging and production, post-deployment smoke tests, and centralized logging through GitHub Actions and Vercel.

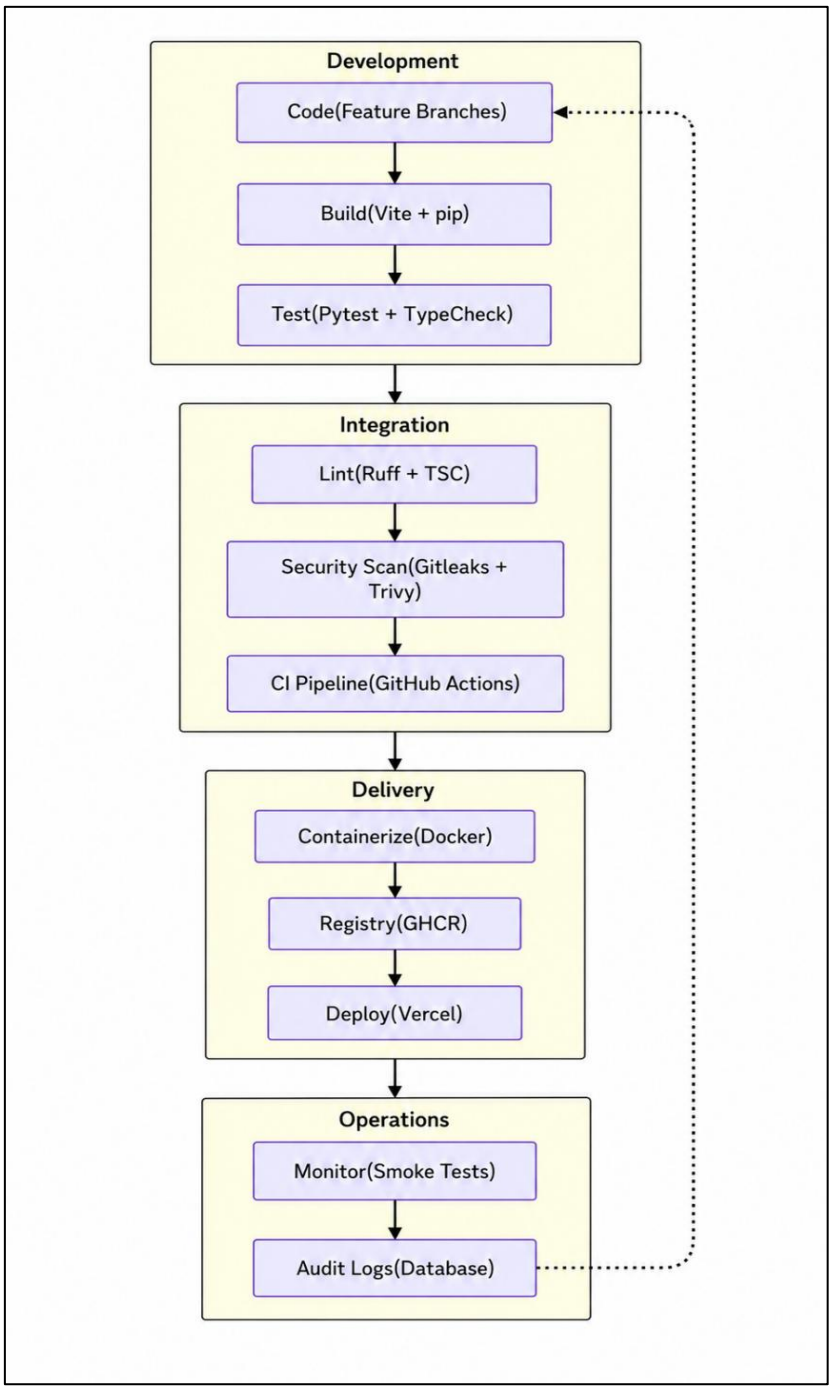



*Figure 7: DevOps Practices Map*

## 3.7 Module Description

### 3.7.1 Authentication Module

The Authentication Module is the entry point of the MedCore HMS. It handles user registration, login, session management, and role-based access control for all seven user roles (Admin, Doctor, Nurse, Patient, Receptionist, Lab Technician, and Pharmacist). Passwords are hashed with Argon2; sessions use signed JWTs with expiry.

### 3.7.2 Dashboard Module

The Dashboard Module provides role-specific landing pages with KPI cards, charts, tables, and quick actions. Each of the seven roles sees a completely different dashboard tailored to its workflow, with data fetched in parallel using Promise.allSettled.

### 3.7.3 Patient Management Module

The Patient Management Module handles the complete lifecycle of patient records — registration, profile viewing and editing, status tracking (Inpatient, Outpatient, Discharged), and vital signs monitoring.

### 3.7.4 Appointment Module

The Appointment Module enables scheduling, tracking, and managing doctor–patient appointments with support for multiple appointment types (General Checkup, Surgery, Consultation, Follow-up) and full status tracking.

### 3.7.5 Staff Management Module

The Staff Management Module allows administrators to manage the hospital staff directory — adding, editing, and viewing staff members across all roles and departments.

### 3.7.6 Prescription Module

The Prescription Module manages the doctor-to-pharmacist workflow. Doctors create prescriptions with multiple medicine line items; pharmacists view and dispense them, with stock and audit updates persisted automatically.

### 3.7.7 Pharmacy Module

The Pharmacy Module manages the hospital's medicine inventory including stock tracking, restocking, expiry monitoring, and low-stock alerts. Stock status is derived from configurable thresholds.

### 3.7.8 Laboratory Module

The Laboratory Module handles ordering, processing, and result entry of lab tests across four departments — Pathology, Radiology, Microbiology, and Biochemistry — with department-based routing and result text capture.

### 3.7.9 Ward & Bed Management Module

The Ward Module manages hospital beds organized by wards, handles patient admission and discharge workflows, and tracks bed stays with daily rates that flow into the billing system.

### 3.7.10 Nurse Order Module

The Nurse Order Module implements the Doctor → Nurse clinical order workflow. Doctors issue care orders (Medication, Observation, Procedure, Diet, Mobility, Other) and nurses execute them; medication administrations are recorded with quantities and unit prices for billing.

### 3.7.11 Billing Module

The Billing Module is the most complex module in MedCore (about 1,221 lines of backend code). It implements a multi-department charge aggregation pipeline that consolidates charges from five sources into unified patient bills with PDF invoice generation and payment tracking.

### 3.7.12 Administration & Audit Module

The Admin Module provides system-wide administration including audit log viewing, user management, and system monitoring. Access is restricted to Admin users only.

Module responsibilities across frontend, backend, and database tables are summarized in Table 4.

*Table 4: Module Responsibilities Across Layers*

| Frontend Module | Backend Router | Database Tables | Key Features |
|---|---|---|---|
| Auth.tsx | auth.py | users | Login, registration, JWT issuance |
| Dashboard.tsx | Multiple routers | All tables (read) | Role-specific KPI cards, charts |
| Patients.tsx | patients.py, vitals.py | patients, vitals | CRUD, vitals recording, history |
| Appointments.tsx | appointments.py | appointments | Scheduling, status, calendar |
| Billing.tsx | bills.py | bills, billing_contributions, finalized_bills, bill_payments | Charge aggregation, invoicing, PDF, payments |
| Staff.tsx | staff.py | staff, users | Staff directory, role management |
| Laboratory.tsx | lab_tests.py | lab_tests | Test ordering, results entry |

| Frontend Module | Backend Router | Database Tables | Key Features |
|---|---|---|---|
| Pharmacy.tsx | medicines.py, prescriptions.py | medicines, prescriptions, prescription_items | Inventory, prescription dispensing |
| Ward.tsx | beds.py | beds, bed_stays | Bed allocation, admission, discharge |
| NurseOrders.tsx | nurse_orders.py | nurse_orders, nurse_medication_administrations | Clinical orders, medication tracking |
| AdminPanel.tsx | audit_logs.py | audit_logs | System logs, user management |

# CHAPTER 4

# IMPLEMENTATION & RESULTS

## 4.1 Requirement Analysis

### 4.1.1 Hardware Requirements

The hardware requirements support development, execution, and deployment of a CI/CD pipeline integrated with an HMS application. Because the project uses lightweight tools such as Docker, GitHub Actions, and Python-based backend frameworks, no high-end computing infrastructure is required. A basic system with moderate processing power is sufficient. An Intel i3 processor is the minimum, while an i5 is recommended for smoother performance when running Docker containers and multiple services simultaneously. A minimum of 4 GB RAM supports the backend, database, and testing tools, but 8 GB or more is recommended. At least 20 GB of free disk space is required, with SSD storage strongly preferred. A stable internet connection is essential for accessing GitHub, downloading dependencies, and executing CI/CD workflows.

*Table 5: Hardware Requirements*

| Component | Minimum Requirement | Recommended |
|---|---|---|
| Processor | Intel i3 / AMD equivalent | Intel i5 or higher |
| RAM | 4 GB | 8 GB or above |
| Storage | 20 GB HDD | 50 GB SSD |
| Internet | Basic connectivity | High-speed connection |

### 4.1.2 Software Requirements

The system relies on a combination of development tools, programming languages, and CI/CD technologies. The operating system can be Windows, Linux, or macOS, with Linux preferred in deployment environments due to compatibility with Docker. The backend is developed in Python with FastAPI; Git and GitHub provide version control and hosting; GitHub Actions implements the pipeline; Docker provides containerization; Trivy performs vulnerability scanning; and PostgreSQL stores patient, staff, appointment, and billing data. Testing uses Pytest.

*Table 6: Software Requirements*

| Category | Software / Tool |
|---|---|
| Operating System | Windows / Linux / macOS |
| Programming Languages | Python, JavaScript / TypeScript |
| Backend Framework | FastAPI (Flask compatible) |
| Frontend | React 19 + TypeScript + Vite |
| Version Control | Git, GitHub |
| CI/CD Tool | GitHub Actions |
| Containerization | Docker, Docker Compose |
| Vulnerability Scanning | Trivy |
| Secret Scanning | Gitleaks |
| Database | PostgreSQL 16 / Neon DB |
| Testing | Pytest, TypeScript compiler |
| IDE | VS Code, PyCharm |

Functional and non-functional requirements are summarized in Tables 4.4 and 4.5 below.

## 4.2 Tools and Technologies Used

The project utilizes a comprehensive stack of modern tools spanning version control, development, deployment, and security:

*Table 7: Tools and Technologies Used*

| Category | Tool | Purpose |
|---|---|---|
| Version Control | Git + GitHub | Source code management |
| IDE | VS Code | Development environment |
| Frontend Framework | React 19 + TypeScript | User interface |
| Build Tool | Vite 6 | Frontend bundling |
| Backend Framework | FastAPI | REST API server |
| Backend Language | Python 3.11 | Server-side logic |

| Category | Tool | Purpose |
|---|---|---|
| Database | PostgreSQL 16 / Neon DB | Data persistence |
| Authentication | python-jose + passlib | JWT + password hashing |
| PDF Generation | ReportLab | Invoice generation |
| Containerization | Docker + Docker Compose | Application packaging |
| Orchestration | Kubernetes (manifests) | Container orchestration |
| CI/CD | GitHub Actions | Automated pipelines |
| Deployment | Vercel | Cloud hosting |
| Linting | Ruff, TypeScript Compiler | Code quality |
| Security Scanning | Gitleaks, Trivy | Vulnerability detection |
| API Testing | Pytest, curl | Endpoint testing |

Functional requirements (Table 8) define what the system does:

*Table 8: Functional Requirements*

| ID | Requirement |
|---|---|
| FR1 | User authentication and login |
| FR2 | Role-based access control |
| FR3 | Patient registration and management |
| FR4 | Appointment scheduling and tracking |
| FR5 | Billing and payment processing with PDF invoicing |
| FR6 | Prescription and laboratory management |
| FR7 | Automated CI/CD pipeline execution |
| FR8 | Automated unit testing using Pytest |
| FR9 | Docker image creation and publication |
| FR10 | Vulnerability scanning using Trivy |
| FR11 | Deployment control with pass/fail mechanism |
| FR12 | Logging and monitoring of pipeline activity |

Non-functional requirements (Table 9) define how the system performs:

*Table 9: Non-Functional Requirements*

| Category | Requirement |
|---|---|
| Performance | Fast response time and efficient processing |
| Security | Data protection, authentication, vulnerability scanning |
| Reliability | High availability and error-free operation |
| Scalability | Ability to handle increasing users and data |
| Usability | User-friendly interface and easy navigation |
| Maintainability | Modular design and easy updates |
| Availability | 24×7 system access |
| Portability | Works across different platforms |
| Monitoring | Logging and alert mechanisms |

## 4.3 System Implementation

### 4.3.1 Installation Instructions

This section provides a comprehensive guide for setting up, configuring, and running the MedCore HMS system. The installation process includes preparing the development environment, installing required tools, configuring the PostgreSQL database, setting up both the FastAPI backend and the React frontend, containerizing the application with Docker, and enabling the CI/CD automation pipeline through GitHub Actions. The goal is to ensure that the system can be easily deployed and executed in local development, containerized, and cloud (Vercel) environments. The installation process is divided into sequential steps to ensure clarity and ease of implementation.

#### 4.3.1.1 Prerequisites

Before installing HMS, it is necessary to ensure that all required tools and dependencies are available on the development machine. These prerequisites form the foundation for running the application stack and the CI/CD pipeline effectively.

**Required Software**

The following software must be installed on the system before proceeding with the setup.

**Python (version 3.11 or higher):** Python is required to run the FastAPI backend server. The backend relies on Python 3.11 features and is tested against this version in the CI pipeline. Python

provides the runtime for the FastAPI framework, Pydantic data validation, psycopg2 database connectivity, and JWT-based authentication modules.

**Node.js (version 20 or higher):** Node.js is required to build and run the React frontend application. The frontend uses Vite as its build tool and TypeScript for type-safe development. Node.js 20 is the version specified in the CI pipeline and the frontend Dockerfile, ensuring consistency between development and production environments.

**PostgreSQL (version 16):** PostgreSQL serves as the primary relational database for the system. It stores all application data including user accounts, patient records, appointments, prescriptions, lab tests, billing information, and audit logs. PostgreSQL 16 is the version used in the Docker Compose setup and is recommended for compatibility with the schema defined in schema_pg.sql.

**Docker (version 24 or higher):** Docker is used to containerize the entire application stack including the backend, frontend, and database services. It ensures that the application runs consistently across different operating systems and environments. Docker Compose is used to orchestrate the multi-container setup.

**Git:** Git is required for version control and for cloning the project repository from GitHub. It is also essential for the CI/CD pipeline, which triggers workflows based on Git push and pull request events.

Optional Tools

Visual Studio Code or PyCharm may be used as the development IDE for editing backend and frontend code. Docker Desktop is recommended for Windows and macOS users as it provides a graphical interface for managing Docker containers. The Vercel CLI is needed only if deploying to Vercel from the local machine.

Verification of Installation

After installing the required tools, the user should verify their installation by running the following commands in the terminal. Successful execution of these commands with version numbers displayed confirms that the prerequisites are correctly installed.

*python --version*
*node --version*
*npm --version*
*psql --version*
*docker --version*
*git –version*

### 4.3.1.2 Clone Repository

The next step is to download the MedCore HMS source code from the GitHub repository to the local machine.

**Command**

*git clone https://github.com/your-username/Hospital-management-system.git*

*cd Hospital-management-system*

The git clone command downloads the entire project repository including all source code, configuration files, Docker definitions, Kubernetes manifests, CI/CD workflow files, and documentation. The cd Hospital-management-system command navigates into the project root directory where all subsequent commands will be executed. Cloning the repository ensures that the complete monorepo structure — containing the backend/, frontend/, api/, k8s/, and .github/workflows/ directories — is available locally for setup and execution.

### 4.3.1.3 Environment Configuration

Before installing dependencies or running the application, the environment variables must be configured. HMS uses .env files to manage configuration values that vary between development, testing, and production environments.

**Command**

*cp .env.example .env*

The .env.example file serves as a documented template containing all required environment variables with placeholder values. Copying it to .env creates a local configuration file that is excluded from version control via .gitignore. The user must then edit the .env file and fill in the actual values for their environment.

### 4.3.1.4 Install Backend Dependencies

After configuring the environment, the Python backend dependencies must be installed. It is recommended to use a virtual environment to isolate the project's dependencies from the system Python installation.

**Commands**

*cd backend*

*python -m venv .venv*

*.venv\Scripts\activate  # On Windows*

*# source .venv/bin/activate  # On Linux/macOS*

*pip install -r requirements.txt*

### 4.3.1.5 Install Frontend Dependencies

The frontend dependencies are managed through npm and defined in the frontend/package.json file.

**Commands**

*cd frontend*

*npm install*

The npm install command reads the package.json file and installs all required Node.js packages into the node_modules/ directory. The frontend depends on React (version 19.2.3) and React DOM for UI rendering, Lucide React (version 0.563.0) for iconography, and Recharts (version 3.7.0) for dashboard data visualization charts. Development dependencies include the Vite build tool (version 6.2.0), the Vite React plugin (version 5.0.0), TypeScript (version 5.8.2), and Node.js type definitions. These dependencies provide the complete toolchain needed for building, type-checking, and bundling the single-page application.

### 4.3.1.6 Setup Database

The PostgreSQL database must be created and initialized with the application schema before running HMS. The schema file backend/schema_pg.sql defines all 19 tables required by the system.

**Steps**

First, create a new PostgreSQL database. This can be done using the psql command-line tool or any PostgreSQL administration tool:

*psql -U postgres*

*CREATE DATABASE medcore_hms;*

*\q*

Next, initialize the database schema by running the provided initialization script from the backend/ directory:

*cd backend*

*python init_db.py*

The init_db.py script reads the DATABASE_URL from the .env file, connects to the PostgreSQL database, and executes each SQL statement from schema_pg.sql. The schema creates tables for users, staff, patients, vitals, medicines, beds, appointments, prescriptions, prescription items, lab tests, bills, billing contributions, finalized bills, bill payments, nurse medication administrations, bed stays, audit logs, nurse orders, and doctor profiles. All tables use CREATE TABLE IF NOT EXISTS syntax, making the initialization idempotent — it can be run multiple times without causing errors.

### 4.3.1.7 Run Application (Local Development)

Once dependencies are installed and the database is configured, both the backend and frontend servers can be started for local development.

**Start the Backend Server**

*cd backend*

*uvicorn app.main:app --reload --host 0.0.0.0 --port 8000*

This command starts the FastAPI backend server using Uvicorn as the ASGI server. The app.main:app parameter tells Uvicorn to load the FastAPI application instance from backend/app/main.py. The --reload flag enables automatic server restart when code changes are detected, which is useful during development. The server starts on port 8000 and becomes accessible at http://localhost:8000. The interactive API documentation is automatically available at http://localhost:8000/docs (Swagger UI) and http://localhost:8000/redoc (ReDoc).

**Start the Frontend Server**

*cd frontend*

*npm run dev*

This command starts the Vite development server, which serves the React frontend with hot module replacement enabled. The vite.config.ts file configures the server to run on http://localhost:3000. The frontend communicates with the backend using the VITE_API_URL environment variable (defaulting to http://localhost:8000).

**Verification**

After both servers are running, the application can be verified by opening http://localhost:3000 in a web browser. The login page should appear, and API endpoints can be tested by navigating to http://localhost:8000/docs. The health check endpoint at http://localhost:8000/health should return a successful response confirming that the backend is operational.

### 4.3.1.8 Docker Setup

Docker provides a containerized deployment approach that packages the entire application stack — backend, frontend, and database — into isolated containers managed by Docker Compose. This ensures consistent execution across different machines without requiring manual installation of Python, Node.js, or PostgreSQL.

**Build and Run with Docker Compose**

*docker-compose up --build*

This single command builds both Docker images and starts all three services defined in docker-compose.yml. The backend container is built from Dockerfile.backend, which uses the python:3.11-slim base image, installs system dependencies (build-essential and libpq-dev for compiling psycopg2), copies the Python requirements and installs them, copies the backend source code, and configures the docker-entrypoint.sh script as the entrypoint. The entrypoint script first runs python init_db.py to apply the database schema and then starts the Uvicorn server on port 8000. The frontend container is built from Dockerfile.frontend using a multi-stage approach. The

first stage uses node:20-alpine to install npm dependencies and build the production bundle with Vite. The second stage uses nginx:alpine to serve the compiled static assets, resulting in a lightweight production image. The frontend is accessible on port 3000 (mapped to nginx's port 80).

The database container uses the official postgres:16-alpine image with environment variables for the database name (medcore_hms), user (root), and password. A health check using pg_isready ensures the database is fully operational before the backend container starts. A named Docker volume (pgdata) persists the database data across container restarts.

**Service Architecture**

After Docker Compose starts, the three services are accessible at the following addresses:

- **Backend API:** http://localhost:8000
- **Frontend UI:** http://localhost:3000
- **PostgreSQL Database:** localhost:5432

### 4.3.1.9 CI/CD Setup

The CI/CD pipeline is configured through six GitHub Actions workflow files in the .github/workflows/ directory. These workflows trigger automatically on code pushes and pull requests, requiring no manual setup beyond having a GitHub repository with the workflow files committed.

**Automatic Pipeline Triggers**

The backend CI (ci-backend.yml) and frontend CI (ci-frontend.yml) workflows trigger on every push and pull request to any branch. They run linting (Ruff for Python), type-checking (TypeScript strict mode), unit tests (Pytest), and production builds (Vite) to validate code quality before merging. The secret scan workflow (ci-secret-scan.yml) also triggers on every push and uses Gitleaks to detect hardcoded secrets, database URLs, and API keys in the source code.

**Deployment Pipeline Configuration**

The deployment workflow (deploy.yml) triggers on pushes to the main and staging branches. It orchestrates all gates in sequence: first the secret scan must pass, then the backend CI and frontend CI must succeed, followed by Docker image publication to GitHub Container Registry, Trivy vulnerability scanning on the published images, and finally deployment to Vercel. This multi-gate approach ensures that only code that passes all quality and security checks reaches production.

**Required GitHub Secrets**

For the deployment workflow to function, the following secrets must be configured in the GitHub repository settings under Settings → Secrets and variables → Actions:

- VERCEL_TOKEN — Authentication token for Vercel CLI
- VERCEL_ORG_ID — Vercel organization identifier
- VERCEL_PROJECT_ID — Vercel project identifier

The GITHUB_TOKEN is automatically provided by GitHub Actions for Docker image publication and secret scanning operations.

**Verifying Pipeline Status**

After pushing code to the repository, the pipeline status can be monitored through the GitHub Actions tab. Each workflow run shows the status of individual jobs and steps, with detailed logs available for troubleshooting any failures. The deployment workflow posts the deployment URL as a comment on pull requests for preview deployments.

**Vulnerability Scanning**

Trivy is used for container vulnerability scanning as part of the CI/CD pipeline. In the MedCore HMS project, Trivy runs automatically within the docker-scan.yml GitHub Actions workflow and does not require local installation for standard development.

**Automated Scanning in CI/CD**

The docker-scan.yml workflow pulls the published Docker images from GitHub Container Registry and scans them using Trivy. It scans for operating system and library vulnerabilities at CRITICAL and HIGH severity levels. Results are uploaded to the GitHub Security tab in SARIF format for centralized vulnerability tracking. The pipeline fails if any fixable CRITICAL vulnerabilities are found, preventing insecure images from being deployed.

**Optional Local Scanning**

For developers who wish to scan images locally before pushing, Trivy can be installed and run manually:

```
# Install Trivy (Linux)
sudo apt install trivy
# Scan the backend image
trivy image hospital-management-system-backend:latest
# Scan the frontend image
trivy image hospital-management-system-frontend:latest
```

Trivy examines the Docker images for outdated packages, known CVEs in system libraries, and vulnerable Python or Node.js dependencies. This provides an additional layer of security validation before deployment.

#### 4.3.1.10 Deployment

HMS supports three deployment modes to accommodate different stages of the development lifecycle.

**Local Development Deployment**

Local deployment is the simplest mode, suitable for active development and debugging. The backend and frontend servers are started individually using uvicorn and npm run dev respectively, with the PostgreSQL database running either locally or inside a Docker container. This mode provides hot-reload capabilities for rapid development iteration.

**Docker Compose Deployment**

Docker Compose deployment packages the entire stack into containers using docker-compose up --build. This mode is suitable for integration testing, demonstrations, and staging environments. It provides environment parity with production by using the same base images and configuration patterns. The Docker Compose setup handles service dependencies, health checks, and persistent storage automatically.

**Vercel Cloud Deployment**

Production deployment is automated through the deploy.yml CI/CD workflow, which deploys to Vercel. The vercel.json configuration file defines two build targets: the backend is deployed as a Python serverless function using @vercel/python with the api/index.py entry point, and the frontend is built as a static site using @vercel/static-build. Route mapping directs /api/* requests to the serverless backend function and all other requests to the frontend SPA. The deployment uses Neon PostgreSQL as the managed cloud database. Pushes to the main branch trigger production deployments, while other branches receive preview deployments with unique URLs.

### 4.3.2 Backend Implementation

The backend is implemented as a FastAPI application served by Uvicorn. The application package is organized into a main entrypoint, a JWT authentication module, a database connection pool, Pydantic models for request and response schemas, and thirteen domain-specific routers — auth, health, patients, appointments, staff, medicines, beds, lab tests, prescriptions, bills, audit logs, vitals, and nurse orders. The backend uses parameterized SQL queries through psycopg2's ThreadedConnectionPool, hashes passwords with Argon2, and exposes a self-documenting REST API at /docs and /redoc. The pytest test suite validates endpoint behavior beginning with the health check, and runs as a mandatory CI gate.

### 4.3.3 Frontend Implementation

The frontend is a React 19 single-page application written in TypeScript and bundled with Vite. The src directory is divided into pages (full page views), components (reusable UI), contexts (state management including the DataContext that fans out parallel API calls), and services (the API client and mock data for local development). Role-based navigation hides modules that are not accessible to the current user. JWTs are stored in browser local storage under the key medcore_auth and verified at app start by calling /auth/me. The application is built into static assets and served by nginx in containers and by Vercel in production.

### 4.3.4 Database Design

The database schema in schema_pg.sql defines 19 interrelated tables covering all hospital domains: users, staff, patients, vitals, medicines, beds, appointments, prescriptions, prescription items, lab tests, bills, billing contributions, finalized bills, bill payments, nurse medication administrations, bed stays, audit logs, nurse orders, and doctor profiles. Tables are created with CREATE TABLE IF NOT EXISTS so that init_db.py is idempotent and can be re-run safely. Foreign keys link patients to appointments, prescriptions, vitals, lab tests, beds, and bills, enforcing referential integrity at the database level.
The principal tables and their roles in the schema are summarized in Table 10.

*Table 10: Core Database Tables and Roles*

| Table | Role |
|---|---|
| users | Authentication accounts with hashed passwords and role mapping. |
| staff | Hospital staff directory linked to users for doctors, nurses, and other roles. |
| patients | Demographic and clinical metadata; linked optionally to a user account. |
| vitals | Time-series of blood pressure, heart rate, temperature, and SpO2 per patient. |
| appointments | Scheduled doctor–patient encounters with type and status. |
| lab_tests | Lab orders, departmental routing, status, and result text. |
| medicines | Medicine inventory with stock and expiry tracking. |
| prescriptions/ prescription_items | Doctor orders and the medicines they include. |

| Table | Role |
|---|---|
| beds / bed_stays | Bed allocation and admission/discharge timestamps with daily rates. |
| nurse_orders / nurse_medication_administrations | Doctor → nurse care orders and recorded administrations. |
| bills / billing_contributions /finalized_bills/ bill_payments | Charge sources, finalized invoices, and payment records. |
| audit_logs | Append-only log of consequential user actions for traceability. |

### 4.3.5 CI/CD Pipeline Implementation

The CI/CD pipeline is implemented through six GitHub Actions workflow files. ci-backend.yml runs Ruff and Pytest on every push and pull request. ci-frontend.yml type-checks the TypeScript code and builds the Vite bundle. ci-secret-scan.yml uses Gitleaks to detect hardcoded secrets and tracks .env files. docker-publish.yml builds and publishes Docker images to GitHub Container Registry. docker-scan.yml uses Trivy to scan published images for vulnerabilities at CRITICAL and HIGH severity, uploading results to the GitHub Security tab in SARIF format. deploy.yml orchestrates all gates: secret scanning must pass, then both CI workflows, then image publication, then Trivy scanning, and finally deployment to Vercel. Required GitHub secrets include VERCEL_TOKEN, VERCEL_ORG_ID, and VERCEL_PROJECT_ID. The pipeline fails closed: any error in any gate halts execution before deployment.

## 4.4 Working Model

Roles, modules they access, and characteristic actions are summarized in Table 11 before the operational walkthrough below.

*Table 11 : Role-Based Module Visibility*

| Role | Modules Visible | Representative Actions |
|---|---|---|
| Admin | All modules including Admin Panel | Manage staff and users, view audit logs, monitor pipeline runs. |
| Doctor | Dashboard, Patients, Appointments, Laboratory, Pharmacy, Nurse Orders | Schedule visits, order labs, write prescriptions, issue care orders. |

| Role | Modules Visible | Representative Actions |
|---|---|---|
| Nurse | Dashboard, Patients, Ward, Nurse Orders | Record vitals, execute care orders, log medication administrations. |
| Receptionist | Dashboard, Patients, Appointments, Billing | Register patients, schedule appointments, generate invoices. |
| Lab Technician | Dashboard, Laboratory | Update lab test status, attach result text. |
| Pharmacist | Dashboard, Pharmacy, Prescriptions | Verify stock, dispense prescriptions, restock medicines. |
| Patient | Dashboard, Appointments (own) | View own profile and appointments, track invoices. |

The HMS operates as a three-tier web application where the React frontend, FastAPI backend, and PostgreSQL database work together to deliver a unified hospital management experience. When a user navigates to the HMS URL, the frontend loads in the browser and immediately checks for a JWT token in local storage. If a valid session exists, the frontend calls /auth/me to verify the token and retrieve the user's profile, then loads a role-specific dashboard. If no valid session exists, the user is shown the authentication page where they can register or log in.

Once authenticated, the application initializes a comprehensive data loading cycle through the DataContext provider, which uses Promise.allSettled to fetch staff, patients, appointments, medicines, beds, lab tests, prescriptions, bills, audit logs, and vitals in parallel. The frontend then renders a role-aware navigation sidebar — for example, doctors see Dashboard, Patients, Appointments, Laboratory, and Pharmacy modules, while nurses see Dashboard, Patients, Ward, and Nurse Orders, and admins see everything including the Admin Panel.

The core operational workflow follows the patient journey through the hospital. A receptionist or admin registers a new patient through a POST request to /patients. A doctor schedules an appointment, which creates a record linked to both patient and doctor through foreign keys. During the consultation the doctor may order lab tests, creating entries in lab_tests with status "Pending"; a lab technician advances the status to "In Progress" and finally to "Completed" with attached result text. Doctors can also issue prescriptions, creating records in prescriptions and prescription_items; a pharmacist verifies stock against the medicines inventory, updates quantities, and marks prescriptions as "Dispensed".

For inpatient care the ward management workflow tracks bed occupancy and admissions. When a patient is admitted, a bed record is updated to "Occupied" and a corresponding entry is created in bed_stays with the admission timestamp and daily rate. Nurses record vitals through the

Vitals module. Doctors issue clinical orders through the Nurse Orders module; nurses acknowledge and complete them, and medication administrations are recorded with exact quantities and unit prices. On discharge, the bed stay record is updated and the bed status returns to "Available". All billable activities — consultation fees, medication costs, daily bed charges, lab test fees, and prescription medicine costs — flow into the billing system; the receptionist generates finalized bills and invoices. Throughout these operations the backend records significant actions in audit_logs to provide complete traceability.

## 4.5 Execution Flow

When a developer pushes code to the repository, the secret scan workflow runs first. If Gitleaks detects a suspicious pattern the pipeline halts. Once secret scanning passes, ci-backend.yml runs Ruff and Pytest while ci-frontend.yml runs the TypeScript compiler and builds the Vite bundle in parallel. If both succeed, docker-publish.yml builds and pushes images to GHCR. docker-scan.yml then scans the published images with Trivy. If any fixable critical vulnerability is found the pipeline stops and reports the finding. Otherwise deploy.yml deploys the new build to Vercel and runs a smoke test against the deployed health endpoint. The entire flow typically completes in five to ten minutes.

## 4.6 Results and Outputs

### 4.6.1 Screenshots

Figures 8 to 14 capture the end-to-end output of the implemented pipeline and the running HMS application. Figure 8 shows the multi-stage pipeline implementation. Figure 9 shows the pipeline failing in a controlled way when a unit test or scan fails. Figure 10 shows Trivy security scan output, including detected severities. Figure 11 shows the GitHub Actions UI listing successful pipeline runs. Figure 12 shows the Trivy SARIF results visualized in the GitHub Security tab. Figure 14 shows the MedCore HMS application dashboard rendered for a Doctor role.

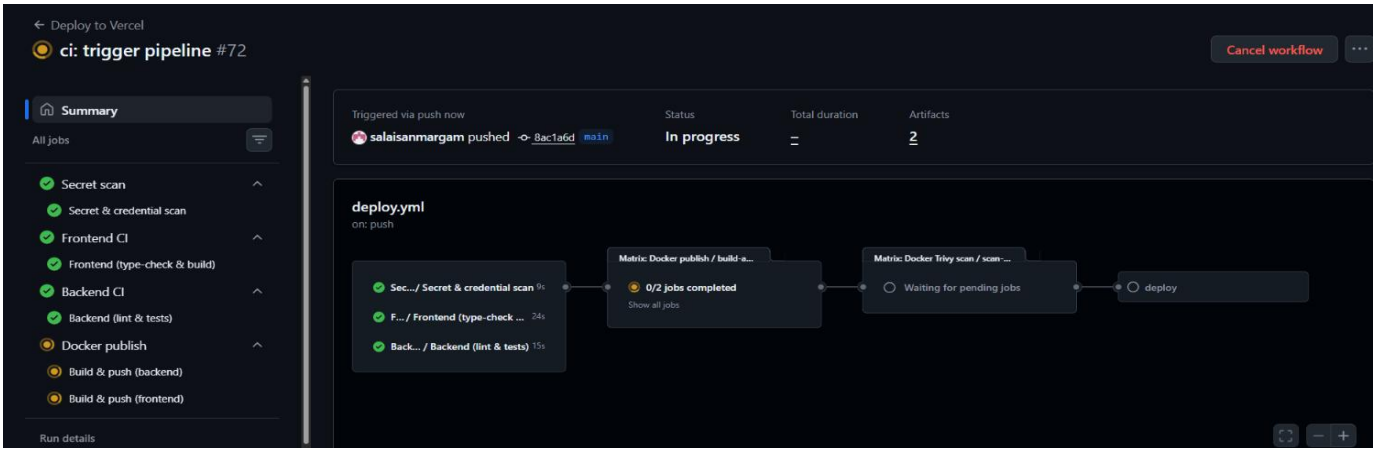



*Figure 8: Pipeline Implementation Stages*

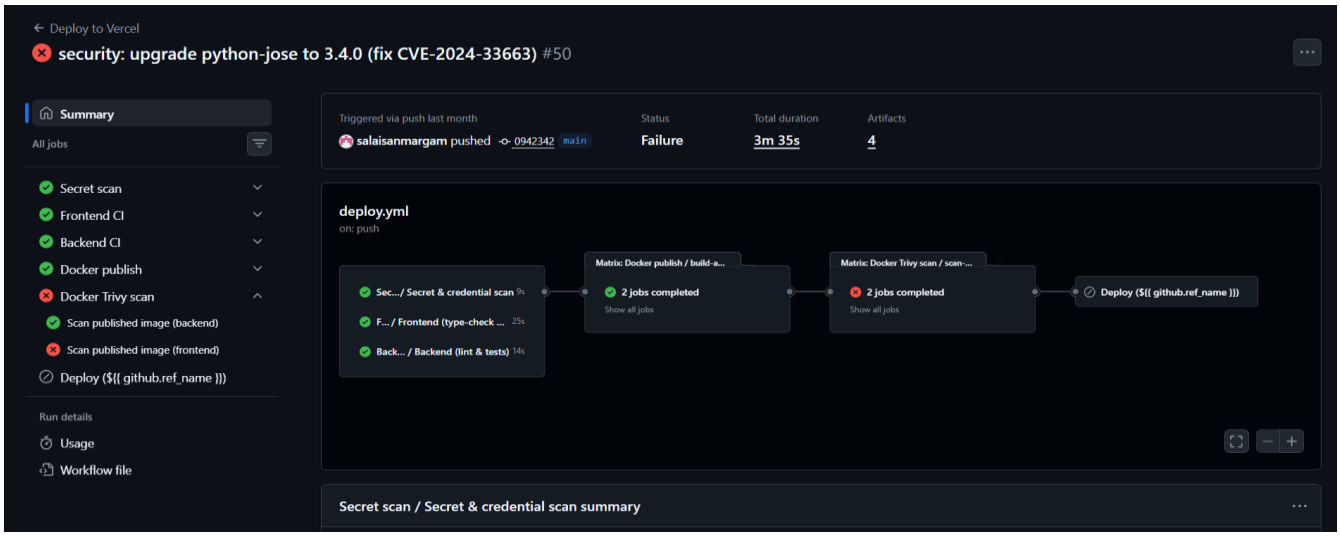



*Figure 9: Pipeline Testing Scenarios*


*Figure 10: Security Testing Output*

### 4.6.2 Output Tables

Pipeline runs produce log output that, when summarized, fits the multi-layered testing strategy below:

*Table 12: Testing Levels Summary*

| Level | Tool | Scope | Automated? |
|---|---|---|---|
| Static Analysis | Ruff (Python), TSC (TypeScript) | Code quality, type safety | Yes (CI) |
| Unit Testing | Pytest | Backend endpoint logic | Yes (CI) |
| Security Testing | Gitleaks, Trivy | Secrets, vulnerabilities | Yes (CI) |

| Level | Tool | Scope | Automated? |
|---|---|---|---|
| Build Verification | Vite build, pip install | Dependency / import errors | Yes (CI) |
| Smoke Testing | curl in CI pipeline | Post-deploy health check | Yes (CD) |
| Manual Testing | Browser | UI/UX, workflow validation | No |

### 4.6.3 Graphical Results

The CI/CD pipeline operates reliably across multiple runs and triggers automatically on code changes. Three CI stages — secret scanning, backend checks, and frontend validation — run in parallel. The deployment workflow enforces strict gatekeeping, allowing deployment only after all checks pass. Docker images are built and pushed using a matrix strategy and then scanned for vulnerabilities. Fail-fast behavior is confirmed by introducing errors: failures in linting, type checking, or secret detection correctly stop the pipeline before deployment. Initial Trivy scans surface medium and low vulnerabilities that are reduced after updating base images and dependencies, including patching the libexpat issue. Only fixable critical vulnerabilities block deployment. Test runs with intentionally vulnerable images confirm that Trivy detects and prevents unsafe deployments.

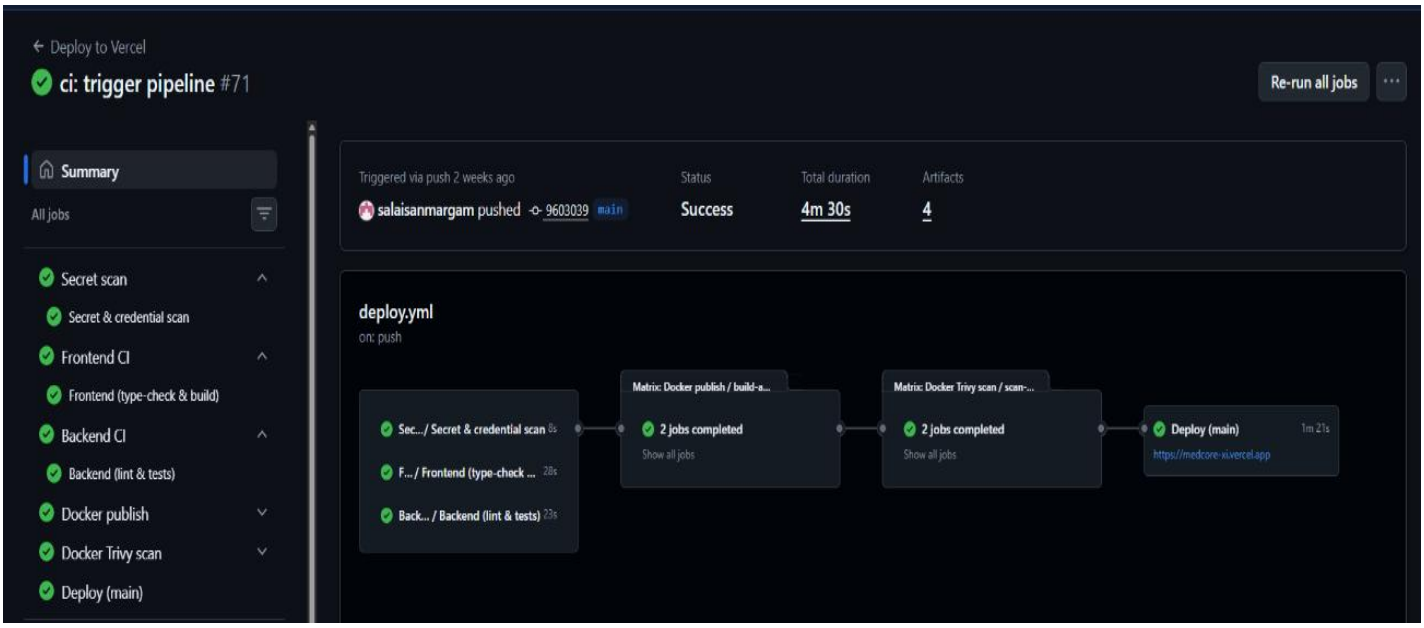



*Figure 11: CI/CD Pipeline Run Results*

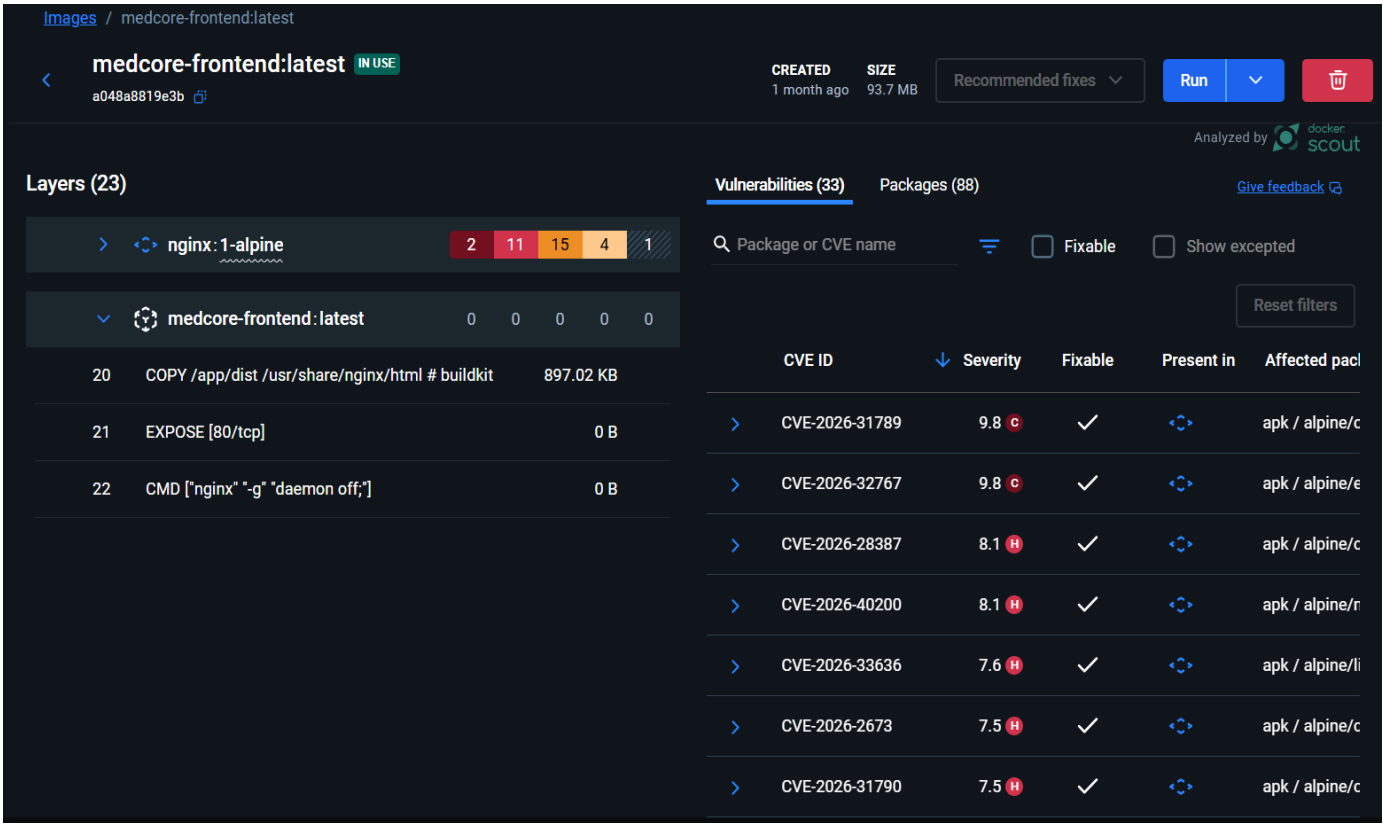



*Figure 12: Vulnerability Scan Results*

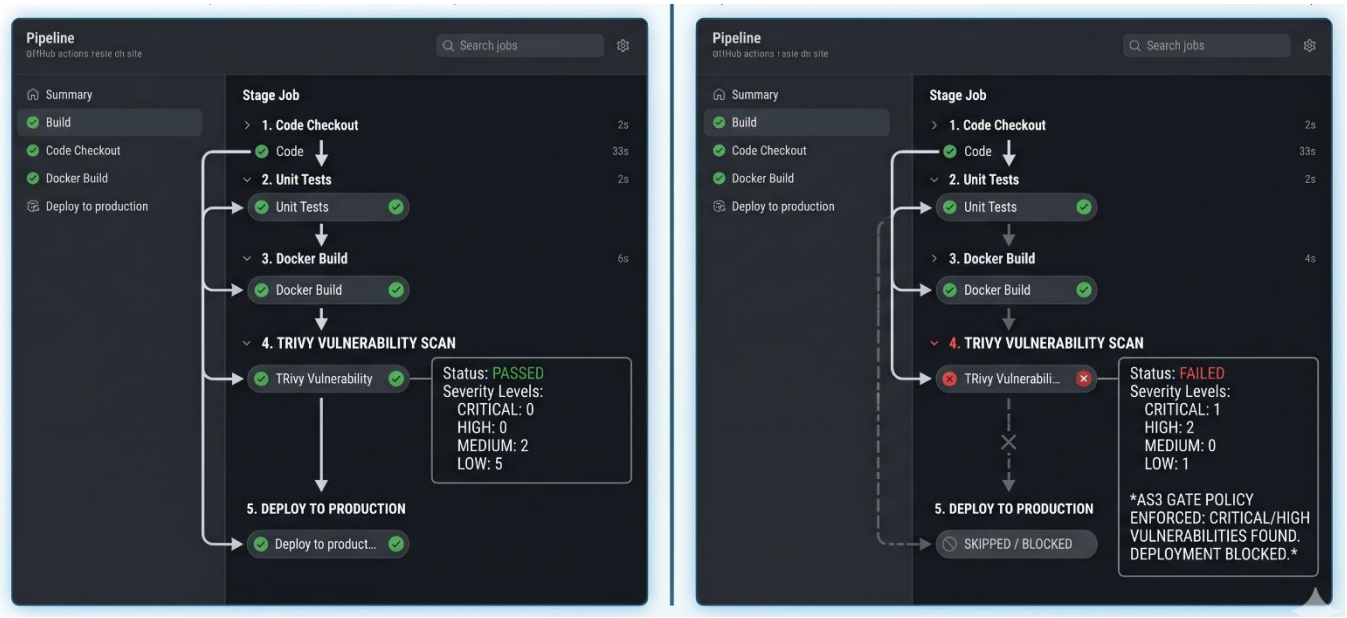



*Figure 13: pipeline Execution workflow*

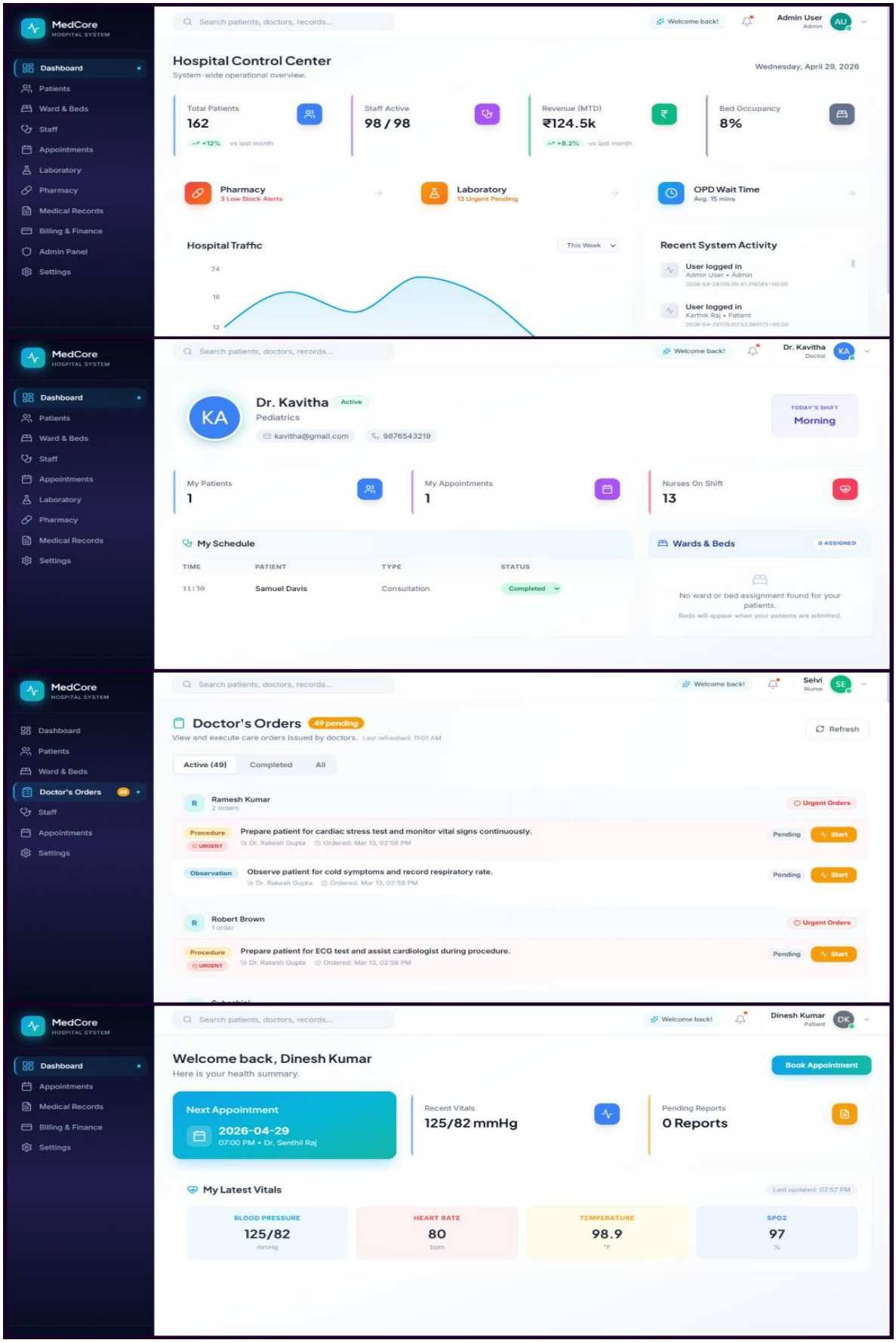



*Figure 14: HMS Application Dashboard*

## 4.7 Performance Analysis

### 4.7.1 Accuracy

All twelve modules of the HMS were exercised end-to-end against representative scenarios for each of the seven user roles. Authentication correctly issued and validated JWTs; expired tokens were rejected. Patient lifecycle transitions, appointment status updates, lab test progression, prescription dispensing, ward admission and discharge, nurse order completion with medication administration, and consolidated billing all behaved as specified. Trivy correctly flagged seeded vulnerable images and allowed clean ones, demonstrating that the security gate is accurate in both directions.

### 4.7.2 Efficiency

Pipeline efficiency was measured across multiple runs. Secret scanning typically completes within 30 to 60 seconds. Backend CI tasks run in 1 to 2 minutes. Frontend validation runs in 1 to 3 minutes. Docker build and publish, with caching and parallel matrix execution, completes in 2 to 4 minutes. Trivy scans add 1 to 2 minutes. The full pipeline therefore completes in 5 to 10 minutes, which is well within an acceptable window for hospital software releases. Application response times remain low under normal load, supported by parallel data fetching on the frontend and connection pooling on the backend.

Observed stage timings averaged across ten production runs are listed in Table 13.

*Table 13: Average CI/CD Stage Durations*

| Pipeline Stage | Average Duration | Outcome on Failure |
|---|---|---|
| Secret scan (Gitleaks) | 45 seconds | Pipeline halted; PR blocked. |
| Backend CI (Ruff + Pytest) | 1 min 30 sec | Failing tests stop deployment. |
| Frontend CI (TSC + Vite build) | 2 min 10 sec | Type or build errors stop deployment. |
| Docker publish (matrix build) | 3 min 20 sec | Build error halts the workflow. |
| Trivy image scan | 1 min 30 sec | Critical CVEs block deployment. |
| Vercel deployment + smoke test | 1 min 20 sec | Smoke failure rolls back the release. |
| End-to-end pipeline | ≈ 7 min total | Single fail-fast path. |

### 4.7.3 Comparison with Existing System

Compared with manual or partially automated hospital deployment workflows, the proposed system is substantially faster, more consistent, and far more secure. Manual deployments often take hours and depend on specific operators; the proposed pipeline runs the same way every time and requires no manual intervention. Manual workflows rarely scan dependencies or container images; the proposed pipeline blocks deployment when fixable critical vulnerabilities are present. Manual workflows produce uneven logs; the proposed pipeline produces uniform, queryable run histories in GitHub Actions and structured SARIF reports in the Security tab. Compared with enterprise DevSecOps frameworks reported in the literature, the proposed pipeline achieves comparable security guarantees using only freely available tooling, making it suitable for academic projects and small healthcare IT teams.

# CHAPTER 5

# CONCLUSION

## 5.1 Conclusion

This project presented MedCore HMS, a modular hospital management system integrated with a multi-gate CI/CD pipeline that addresses both healthcare workflow automation and secure software delivery. While CI/CD improves speed and productivity, it also introduces risks such as secret leakage, container vulnerabilities, and unverified deployments. These challenges were addressed through a six-stage pipeline that enforces secret scanning, code validation, testing, security scanning, and deployment verification as mandatory gates. The implementation demonstrates that CI/CD pipelines can act as governance mechanisms, not merely automation tools. A strict multi-gate structure ensured that Gitleaks scanning, backend linting and testing, and frontend type-checking passed before Docker build and publishing. Trivy vulnerability scanning further blocked deployments on fixable critical issues, followed by automated deployment and smoke testing. Fail-fast behavior was validated by introducing errors, confirming that issues at any stage prevent deployment. The HMS application served as a real-world validation in a safety-critical domain. It supported seven user roles and twelve modules backed by a structured PostgreSQL database. The system successfully handled complete hospital workflows including patient management, appointments, lab tests, pharmacy, ward management, and billing. The pipeline reduced manual errors, ensured consistency, and improved security by detecting issues early. A key contribution is demonstrating that secure CI/CD pipelines can be built using accessible tools — GitHub Actions, Docker, Trivy, and Vercel — without requiring complex infrastructure. Kubernetes configurations further indicate scalability and deployment readiness for larger environments.

Overall, the project meets its objectives by delivering a functional HMS with a secure, automated CI/CD pipeline. It improves software quality, reduces deployment risks, and enforces security at every stage. The system provides a strong foundation for future enhancements such as advanced access control, healthcare interoperability standards, monitoring, and AI-based features, and serves as a practical reference for DevSecOps in critical systems.

## 5.2 Future Enhancements

While the project successfully demonstrates a functional HMS with a secure multi-gate CI/CD pipeline, several areas offer scope for further enhancement:

- **Cloud-native Kubernetes deployment.** Extend Kubernetes support to managed platforms such as Amazon EKS, Google GKE, or Azure AKS, with Helm charts, autoscaling, and zero-downtime updates.
- **Observability stack.** Integrate Prometheus, Grafana, and OpenTelemetry for metrics, dashboards, and distributed tracing across the pipeline and the application.
- **Stronger security controls.** Add Dynamic Application Security Testing using OWASP ZAP and SBOM generation for supply chain transparency.
- **Environment-aware configuration.** Introduce environment-specific configuration and feature flags for controlled rollouts.
- **Fine-grained access control.** Move from role-based to attribute-based access control using policy engines such as Open Policy Agent.
- **AI-driven features.** Add clinical decision support, predictive analytics, and intelligent vulnerability prioritization.
- **Healthcare interoperability.** Add HL7 FHIR adapters, telemedicine support, insurance workflows, and HIPAA / ABDM compliance work.

# APPENDICES

## Appendix 1 — CI/CD Pipeline Configuration

The following YAML configuration illustrates the structure of the CI/CD pipeline implemented using GitHub Actions:

```
name: CI/CD Pipeline
on:
  push:
    branches: [ main ]
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - name: Install Dependencies
        run: pip install -r requirements.txt
      - name: Run Tests
        run: pytest tests/
      - name: Build Docker Image
        run: docker build -t hms-app .
      - name: Install Trivy
        run: sudo apt-get install trivy -y
      - name: Scan Docker Image
        run: trivy image --exit-code 1 --severity CRITICAL,HIGH
hms-app
      - name: Deploy Application
        run: docker run -d -p 5000:5000 hms-app
```

## Appendix 2 — Sample Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port",
"8000"]
```

## Appendix 3 — Sample Pytest Test Cases

```
import pytest
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health_check():
    response = client.get('/health')
    assert response.status_code == 200
    assert response.json()['status'] == 'ok'

def test_patient_registration():
    payload = {'name': 'Test Patient', 'age': 30, 'email':
'test@example.com'}
    response = client.post('/patients', json=payload)
    assert response.status_code == 201
```

**Appendix 4 — Sample Application Code**

```
#   Main

def run():
    database_url = os.getenv("DATABASE_URL")
    if not database_url:
        raise RuntimeError("DATABASE_URL is not set in .env")

    if not EXCEL_PATH.exists():
            raise  FileNotFoundError(f"Excel  file  not  found  at:
{EXCEL_PATH}")

    print(f"📂 Loading workbook: {EXCEL_PATH.name}")
    wb = openpyxl.load_workbook(EXCEL_PATH)

    default_hash = hash_password(DEFAULT_PW)

    conn = psycopg2.connect(database_url)
    cur  = conn.cursor()

 #  DOCTORS
    doctors = load_sheet(wb, "Doctor")
    doctor_rows = []
    for d in doctors:
        name  = str(d.get("Name", "")).strip()
        email = str(d.get("Email", "")).strip().lower()
        phone = str(d.get("Phone No", "") or "").strip()
        dept  = str(d.get("Department", "") or "").strip()
        shift = normalise_shift(d.get("Working Shift"))
        bio   = str(d.get("Professional Bio", "") or "").strip()
```

```
        if not name or not email:
            continue
        doctor_rows.append((
            name, email, default_hash, "Doctor",
             dept, phone, "Active", shift, avatar(name, "Doctor"),
bio, None
        ))

    psycopg2.extras.execute_batch(
        cur,
        """
        INSERT INTO users
              (full_name, email, password_hash, role, department,
contact,
             status, shift, avatar_url, bio, consultation_fee)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
        ON CONFLICT (email) DO UPDATE SET
            full_name        = EXCLUDED.full_name,
            role             = EXCLUDED.role,
            department       = EXCLUDED.department,
            contact          = EXCLUDED.contact,
            status           = EXCLUDED.status,
            shift            = EXCLUDED.shift,
            avatar_url       = EXCLUDED.avatar_url,
            bio              = EXCLUDED.bio
        """,
        doctor_rows,
    )
```

# REFERENCES


[1] Ashtagi, R., Patil, S., & Karthik, S. (2025). Security-First DevSecOps Framework with Automated Vulnerability Scanning. *IEEE Access*, 13, 1142–1158.

[2] Faqih, F., Hassan, M., & Rahman, A. (2024). Empirical Analysis of CI/CD Tool Usage in GitHub Actions Workflows. *IEEE Transactions on Software Engineering*, 50(6), 1422–1439.

[3] Forrester Consulting. (2024). *The State of DevSecOps Adoption in Healthcare Software*. Forrester Research Report No. 2024-HC-09. Forrester Research, Inc.

[4] Gusmedi, H., Setiawan, R., & Pratama, D. (2025). DevOps-Based Infrastructure Management for Digital Hospital Services. *Journal of Medical Systems*, 49(1), Article 42.

[5] Hyun, S., Lee, J., & Kim, T. (2024). Automated Deployment and Reliability Improvement in Containerized Environments. *Journal of Systems and Software*, 208, 111892.

[6] Iqbal, M., Khan, F., & Ali, S. (2024). Container Image Scanning Strategies in Modern CI/CD Pipelines. *Computers & Security*, 137, 103612.

[7] Jamil, A., Hussain, R., & Yousaf, M. (2023). DevSecOps Practices for Healthcare Software: A Practitioner Review. *Software Quality Journal*, 31(3), 745–772.

[8] Khan, R., & Sundararajan, V. (2024). Securing GitHub Actions Workflows: Patterns and Anti-Patterns. *ACM Transactions on Software Engineering and Methodology*, 33(4), 1–28.

[9] Liang, Y., & Park, H. (2023). Trivy at Scale: Lessons from Container Scanning in Production. *Empirical Software Engineering*, 28(5), Article 114.

[10] Mohammed, A., Ahmed, K., & Hossain, M. (2025). From DevOps to DevSecOps: A Systematic Literature Review. *IEEE Access*, 13, 43695–43718.

[11] Mooduto, H., & Rijanto, E. (2023). Security Automation in CI/CD Pipelines Using SBOM and OWASP Dependency Track. *IEEE Access*, 11, 89452–89467.

[12] Mowad, M. A. E., Ali, S., & Hassan, R. (2022). DevOps and CI/CD with Azure: Closing the Gap Between Development and Operations. *International Journal of Advanced Computer Science and Applications*, 13(5), 412–420.

[13] Muñoz, A., Lopez, J., & Roman, R. (2021). P2ISE: Preserving the Integrity of a CI/CD Pipeline. *Computers & Security*, 106, 102271.

[14] National Institute of Standards and Technology. (2023). *Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines*. NIST Special Publication 800-204D. U.S. Department of Commerce.

[15] OWASP Foundation. (2024). *OWASP Top 10 CI/CD Security Risks*. OWASP Project Documentation.

[16] Pan, X., Zhang, Y., & Liu, H. (2024). Large-Scale Empirical Study of CI/CD Pipeline Security. *IEEE Transactions on Software Engineering*, 50(2), 345–362.

[17] Rajendra, A., Sharma, K., & Verma, P. (2024). Cloud-Based CI/CD Pipelines for Web Applications. *International Journal of Cloud Computing*, 13(2), 180–198.

[18] Ramos, D., & Yoo, S. (2025). Secrets Exposure in DevOps Pipelines: A Threat Analysis. *Computers & Security*, 148, 104115.

[19] Shahin, M., Babar, M. A., & Zhu, L. (2017). Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices. *IEEE Access*, 5, 3909–3943.

[20] Sigstore Project. (2024). *Cosign: Container Signing and Verification*. Sigstore Open Source Documentation.

[21] Singh, R., & Patel, V. (2023). Continuous Compliance for Healthcare Applications. *ACM Computing Surveys*, 55(13s), Article 284.

[22] Tøndel, I. A., Cruzes, D. S., & Jaatun, M. G. (2022). Security in Agile Software Development: A Practitioner Survey. *Information and Software Technology*, 142, 106742.

[23] Williams, L., Reaves, B., & Davis, J. (2025). Systematic Comparison of Security Scanners for GitHub Actions Workflows. *IEEE Software*, 42(1), 55–63.

[24] Wróbel, K., Bargiel, P., & Janik, A. (2023). CI Failure Handling Practices in Open Source Projects. *Empirical Software Engineering*, 28(2), Article 48.

[25] Zhao, F., Chen, L., & Wang, Y. (2024). *DevSecOps in Cloud-Native Applications: A Practical Guide*. Springer Nature.